

Final Report: RF Signal Classification via Deep Learning

Project: RF Signal Classification from HackRF Spectrograms
Template: 3.2 - Deep Learning Breast Cancer Detection

Introduction (896/1000 words)

What problem is this project solving?

Convolutional neural networks (CNNs) can classify radio frequency (RF) modulation types from spectrogram images produced by software-defined radios (SDRs) (Waqas et al., 2023). In this project I extend those methods and evaluate their sim-to-real performance on consumer-grade hardware, while testing the approach beyond controlled laboratory settings.

The project's motivation is practical. Autonomous systems, such as drones and mobile sensors, need to recognise unknown signals for machine-to-machine and environment communication. Often, physical and economic constraints limit these platforms to low-cost radios. However, prior over-the-air (OTA) validation relied on laboratory-grade radios (O'Shea et al., 2018). Thus, the central question is whether spectrogram-based CNN classification, trained on the standardised RadioML 2018.01A dataset, holds up on live captures from consumer-grade hardware where receiver impairments degrade the signal. Within the literature reviewed for this project, I found no study that evaluates spectrogram-based CNN modulation classification on consumer-grade SDR hardware. Testing this gap on a low-cost HackRF One SDR (Great Scott Gadgets, 2026) is this project's primary goal. That question has two separable halves, the receiver and the waveform, and this report answers them differently: the consumer radio proved sufficient to receive and represent a live broadcast, while the model trained on synthetic signals still failed on it, because a real transmission carries a richer signature than the synthetic class it was trained on.

Background and context

Automatic Modulation Classification (AMC) identifies a signal's modulation scheme without prior transmitter knowledge. Modulation encodes information on a carrier by changing its amplitude, frequency, or phase. The project's input is complex **I/Q** data: in-phase and quadrature components that jointly describe a signal's amplitude and phase. **Signal-to-Noise Ratio (SNR)** compares signal power with noise power in decibels, higher SNR means a cleaner observation. AMC analyses these changes to determine the modulation type and is widely used in cognitive radio, electronic warfare, spectrum monitoring, and 5G/6G networks (Zheng et al., 2025; Chennagiri, Sehgal and Ravinder, 2024; Huynh-The et al., 2021).

Early AMC relied on hand-crafted statistical features. Traditional likelihood-based and feature-based methods require prior channel knowledge and degrade sharply at low SNR (Zheng et al., 2025; Chennagiri, Sehgal and Ravinder, 2024). Newer research applies deep learning to I/Q components (Zheng et al., 2025; Chennagiri, Sehgal and Ravinder, 2024). O'Shea et al. (2018) showed CNNs learn useful features directly from raw I/Q data and released unmodified radio emissions with ground-truth modulation labels. I use this dataset to enable comparison with existing research. I also convert I/Q to spectrograms using the Short-Time Fourier Transform (STFT), which measures frequency content in short overlapping time windows. Waqas et al. (2023) found STFT spectrograms effective as CNN input, testing six architectures across 11 modulation types and reaching up to 91.2% accuracy at $\text{SNR} \geq 5$ dB. The baseline experiment covers all 26 RadioML SNR levels, the final five-channel model is evaluated only at $\text{SNR} \geq -2$ dB, followed by sim-to-real testing on consumer SDR hardware.

AMC is challenging due to channel noise, multipath fading, timing errors, Doppler offsets (frequency shifts from motion), and other physical effects. Distinctions between modulation types,

such as phase-shift keying (PSK) and quadrature amplitude modulation (QAM) variants, become subtle at low SNR, where likelihood and feature-based classifiers lose accuracy (Zheng et al., 2025; Huynh-The et al., 2021).

For real-world capture, I used a HackRF One SDR (1 MHz-6 GHz, Great Scott Gadgets, 2026) with GNU Radio for RF visualisation (GNU Radio Foundation, 2026). For model building and benchmarking, I used DeepSig's RadioML 2018.01A dataset (O'Shea et al., 2018), containing labelled I/Q data across 24 modulation formats. This dataset combines synthetic channel simulation with OTA recordings, stored as complex floating-point values in Hierarchical Data Format version 5 (HDF5). It includes 26 SNR levels from -20 dB to +30 dB (2 dB steps) and, per the dataset documentation, 4,096 frames per modulation/SNR pair, each consisting of 1,024 complex I/Q samples shaped (1024, 2).

To keep training manageable, I selected eight of the 24 modulation classes: 4ASK, BPSK, QPSK, 16PSK, 16QAM, FM, AM-DSB-WC, and 32APSK, each named in full in the Design chapter. This subset covers analogue and digital schemes across amplitude, frequency, and phase variations. I selected 16QAM instead of higher-order schemes such as 128APSK or 256QAM to limit low-SNR ambiguity while retaining geometric variety. Even hybrid CNN+Support Vector Machine (SVM) models struggle with same-family schemes such as 16QAM and 64QAM (Abd-Elaziz, Abdalla and Elsayed, 2023), a pattern echoed by confusion among other complex digital schemes such as 8PSK and 32QAM (Nasir, Sheikh and Malik, 2025).

The approach applies CNN-based image classification to STFT spectrograms and related feature maps derived from I/Q samples.

Project goal

The project follows Template 3.2, Deep Learning Breast Cancer Detection, and the project number used throughout this report is 3.2. It tested whether a CNN trained on spectrogram images could classify RF modulation when moving from simulation to real-world use on a low-cost HackRF One SDR. It developed a CNN following Chollet and Watson's (2026) workflow, using RadioML 2018.01A as the training and evaluation basis. The study measured synthetic performance and tested reliability on a live OTA capture, establishing practical limits for deployment on affordable radios. Stated plainly, this project's specific contribution is twofold: a five-channel spectrogram representation that was added only after a measured baseline failure mode showed time-frequency energy alone could not separate closely related digital schemes, and a direct test of that representation on consumer-grade rather than laboratory-grade OTA hardware, which the reviewed literature does not report.

Literature Review (1965/2500 words)

From Traditional Methods to Deep Learning

AMC methods divide into likelihood-based, feature-based, and deep-learning approaches (Chennagiri, Sehgal and Ravinder, 2024; Zheng et al., 2025).

Likelihood-based methods treat modulation classification as a hypothesis test over candidate schemes. In reviewing prior likelihood-based methods, Abd-Elaziz, Abdalla and Elsayed (2023) summarise that a Maximum-Likelihood detector reaches 80% accuracy at 0 dB SNR and approaches 100% at 20 dB SNR for six modulation formats, whereas the Hybrid Likelihood Ratio Test (HLRT) attains only 31% accuracy at -15 dB SNR. These results expose a key drawback of likelihood-based methods: they require accurate prior channel knowledge, and computational load grows sharply with candidate modulations, making them unsuitable for non-cooperative scenarios where the receiver lacks transmitter information.

Feature-based methods extract handcrafted features (instantaneous amplitude/phase, higher-order cumulants, cyclostationary features, wavelet coefficients) for classifiers such as Support Vector Machines, decision trees, or K-Nearest Neighbors (Zheng et al., 2025; Chennagiri, Sehgal and Ravinder, 2024). Zheng et al. (2025) note these features are interpretable but degrade sharply at low SNR. Chennagiri, Sehgal and Ravinder (2024) find feature-based methods suitable under data or latency constraints but inferior to deep learning when labelled data is available.

The deep learning shift followed RadioML 2018.01A release and O’Shea et al.’s (2018) demonstration that CNNs learn discriminative features from raw I/Q without feature engineering (Zheng et al., 2025; Huynh-The et al., 2021). The dataset covers 24 modulation types across a wide SNR range and serves as this project’s benchmark. Huynh-The et al. (2021) surveyed convolutional, recurrent, long short-term memory, and hybrid neural network architectures, finding deep learning outperforms classical methods under channel impairments, especially at low SNR. Their survey catalogued image-based approaches, including constellation and spectrogram representations, alongside raw I/Q methods. Waqas et al. (2023) subsequently addressed STFT spectrograms.

Signal Representations. What Goes Into the Classifier?

Three signal representations appear across the surveyed papers: raw I/Q sequences, visual diagrams (eye diagrams, constellation diagrams), and spectral images (STFT spectrograms, wavelet scalograms).

Raw I/Q sequences are the most common input. O’Shea et al. (2018) showed that ResNet and Visual Geometry Group (VGG) style CNNs on 2×1024 I/Q frames from RadioML 2018.01A reach $\sim 95\%$ at high SNR across 24 modulation types. Abd-Elaziz, Abdalla and Elsayed (2023) used a custom CNN with parallel asymmetric kernels and skip connections, achieving 86.1% at -2 dB SNR on nine modulation schemes. Nasir, Sheikh and Malik (2025) applied established architectures (ResNet, DenseNet, MobileNet, AlexNet and Inception-ResNet) to adapted raw-I/Q inputs and then compared CNN and SVM classifiers.

Eye diagrams (Rehman, Abbas and Shehzad, 2025) fold I/Q components over time into 2D images capturing interference, jitter, and noise. This representation is more noise-resilient than constellation diagrams as it accumulates multiple traces rather than a single scatter plot. ResNet-18, ResNet-50, and MobileNetV2 achieve high accuracy across 14 modulation types (Rehman, Abbas and Shehzad, 2025). However, eye diagram generation requires precise symbol timing synchronisation (raised cosine filtering with known roll-off and samples-per-symbol parameters, stated explicitly by Samarkandi et al., 2023), which may be inaccessible in non-cooperative scenarios lacking prior synchronisation data.

Spectral images (Waqas et al., 2023) convert I/Q sequences to STFT spectrograms via a resolution transformation that cuts computational overhead by 99.61%. Comparing six CNNs across 11 modulation types, they achieved 91.2% accuracy at $\text{SNR} \geq 5$ dB. The multi-scale architectures (InceptionResNet-V2, Inception-V3) outperformed single-scale networks. Zheng et al. (2025) discuss the temporal-frequency resolution trade-off of STFT and wavelet transforms, comparing window choices rather than evaluating the two representations against each other. Nasir, Sheikh and Malik (2025) note that Fast Fourier Transform and Welch power-spectrum visualisations yield comparatively lower average accuracy for multilevel amplitude-shift-keying signals.

STFT spectrograms serve as CNN input here, extending Waqas et al. (2023) in three ways: evaluation on the standardised RadioML 2018.01A benchmark, a full-range baseline followed by final-model evaluation at $\text{SNR} \geq -2$ dB rather than ≥ 5 dB only, and assessment of live HackRF One captures. The study measures how well a model trained in simulation generalises to physical hardware.

What the Previous Work Establishes

The literature suggests that AMC performance depends on the relationship between the representation and the assumptions available to the receiver, not simply on classifier depth. Conventional likelihood-based methods can be statistically well-founded, but require estimates of signal and channel parameters that are often unavailable in non-cooperative sensing. For example, the comparison summarised by Abd-Elaziz, Abdalla and Elsayed (2023) reports 80% for maximum-likelihood detection at 0 dB across six schemes, but only 31% for a hybrid likelihood-ratio test at -15 dB. O’Shea et al. (2018) provide a useful intermediate benchmark: their higher-order-statistics and gradient-tree baseline uses compact expert features, whereas their CNN learns features directly from I/Q. The comparison is important because it frames deep learning as a change in where domain knowledge is placed, rather than proof that learned features are automatically robust to every channel.

The representation studies also provide a specific reason to combine views of a signal. Samarkandi et al. (2023) compare constellation diagrams, eye diagrams, and ambiguity functions for 8QAM, 16QAM, and 32QAM under SNR from -20 to 10 dB. Their results show that constellation and eye diagrams remain substantially stronger than the ambiguity function in their experiment, and that combining constellation and eye representations improves performance at difficult SNRs. However, their generated three-class dataset and raised-cosine filtering assumptions do not establish performance on RadioML 2018.01A or on unsynchronised OTA captures. The contribution is therefore methodological: complementary representations can expose information that one image type omits, but the useful combination must be tested under the target dataset and receiver conditions.

This distinction motivates the design. An STFT preserves when and where energy occurs in frequency, which is useful for FM and AM, but it averages away much of the angular geometry that separates PSK, QAM, and APSK. The five-channel input combines STFT-derived dynamics with an I/Q density histogram and a phase-time histogram. This responds to the baseline confusion matrix. The project tests two hypotheses: explicit geometric information improves separation of the digital classes, and this improvement may still fail under hardware and waveform differences.

Comparison of Prior Work

Table 1a. Comparison of AMC study inputs and architectures.

Study	Input	Architecture
O’Shea et al. (2018)	Raw I/Q	ResNet + VGG-style CNN
Abd-Elaziz et al. (2023)	Raw I/Q	Custom CNN (asymmetric kernels, skip connections)
Waqas et al. (2023)	STFT spectrogram	Six CNNs incl. InceptionResNet-V2, Inception-V3
Rehman et al. (2025)	Eye diagram	ResNet-18/50, MobileNetV2
Nasir et al. (2025)	Adapted raw I/Q	CNN+SVM (ResNet, DenseNet, MobileNet, AlexNet)
This project	Five-channel STFT	Custom CNN (4 conv. blocks + residual)

Table 1b. Comparison of AMC study datasets and results.

Study	Dataset	Classes	Best accuracy
O'Shea et al. (2018)	RadioML 2018.01A	24	~95% at high SNR
Abd-Elaziz et al. (2023)	Simulated additive white Gaussian noise/Rician	9	99.8% at 10 dB
Waqas et al. (2023)	RML2016.10a (public)	11	91.2% above 5 dB
Rehman et al. (2025)	Custom synthetic	14	High (−20 to +30 dB)
Nasir et al. (2025)	HISARMOD2019.1	26	Up to 100% (selected)
This project	RadioML 2018.01A	8	93.8% at SNR ≥ −2 dB

Analysis of Tables 1a and 1b reveals common patterns in prior research. Most studies report headline accuracy on synthetic or benchmark data, so class- or SNR-specific figures approaching 100% in Nasir, Sheikh and Malik (2025) should not be interpreted as overall real-world accuracy: that study uses the HISARMOD2019.1 benchmark and identifies real-world testing as future work. O'Shea et al. (2018) is the important exception in using the same RadioML 2018.01A benchmark and adding OTA measurements, although those measurements use laboratory-grade Universal Software Radio Peripheral (USRP) B210s rather than low-cost receivers typical in deployment. Reported accuracies are not directly comparable because the datasets, modulation sets, input representations, and SNR ranges differ. The literature therefore lacks evaluation of spectrogram-based classification on consumer-grade hardware, the gap this project targets.

The studies nevertheless support a more specific conclusion than “deeper networks perform better”. O'Shea et al. (2018) show that residual and VGG-style CNNs can learn useful features from raw I/Q on a shared benchmark, while Waqas et al. (2023) show that multi-scale CNNs can exploit structure in spectrograms. These findings concern different representations, so they do not establish that a larger architecture will improve spectrogram classification. Conversely, Samarkandi et al. (2023) provide evidence that combining complementary views can help difficult digital classes, but their dataset and synchronisation assumptions limit direct transfer to this project. Taken together, the strongest unresolved factor is representation: a spectrogram preserves temporal-frequency energy but may discard constellation geometry. The study therefore tests representation fusion before increasing model complexity, then checks whether any simulated-data improvement survives capture by a consumer SDR. That separates three claims that are often conflated in prior work: performance on a benchmark, the value of an input representation, and robustness to hardware conditions.

Persistent Gaps and the Contribution of This Project

Read together, the four limitations below combine into a single testable claim rather than four unrelated observations: existing spectrogram and representation studies do not evaluate on consumer-grade hardware, do not report a full-range baseline alongside an SNR-filtered result on the same standardised benchmark, do not cover a comparable class set, and do not isolate architectural choices under matched constraints. This project responds to all four with one coherent design rather than addressing them in isolation. The OTA test on a HackRF One directly addresses the absence of consumer-hardware evaluation. Adding I/Q-density and phase-time channels targets the geometric information that STFT alone omits, measured through the change in digital-class F1 and the confusion matrix. A balanced RadioML held-out split with results reported for both the full-range baseline and the SNR-filtered model enables comparison across the two data regimes. A custom CNN trained from scratch was chosen over pretrained

backbones and evaluated against transfer-learning runs under the same CPU and dataset constraints.

Limited real-hardware evaluation. O’Shea et al. (2018) achieved 95.6% accuracy by training directly on USRP B210 OTA captures at 900 MHz, and approximately 87% when transferring a synthetic model to OTA data. However, the USRP B210 is laboratory-grade hardware. It is unclear how well these results generalise to consumer-grade hardware. Common receiver impairments including phase noise, DC offset, I/Q imbalance, and Analogue-to-Digital Converter (ADC) non-linearity can degrade AMC accuracy. The other surveyed studies use simulated or benchmark data (Zheng et al., 2025; Abd-Elaziz, Abdalla and Elsayed, 2023; Nasir, Sheikh and Malik, 2025; Rehman, Abbas and Shehzad, 2025; Waqas et al., 2023). Zheng et al. (2025) identifies hardware and deployment variability as an open challenge. OTA evaluation here extends to consumer hardware (HackRF One).

Spectrogram benchmarking across SNR. Waqas et al. (2023) evaluate STFT spectrograms on 11 modulation types at $\text{SNR} \geq 5$ dB using the RML2016.10a dataset, while O’Shea et al. (2018) cover all 24 RadioML 2018.01A classes but use only raw I/Q. Within the literature reviewed for this project, I found no study applying STFT spectrograms to RadioML 2018.01A in this combination of a full-range baseline and a final five-channel model evaluated at $\text{SNR} \geq -2$ dB. Both regimes are reported separately.

Fragmented benchmark coverage. Each surveyed paper uses a different dataset. Abd-Elaziz, Abdalla and Elsayed (2023) tested nine modulation schemes, Waqas et al. (2023) eleven, and Rehman, Abbas and Shehzad (2025) fourteen. Nasir, Sheikh and Malik (2025) uses the 26-class HISARMOD2019.1 benchmark, comparable in class count to O’Shea et al.’s (2018) RadioML 2018.01A. The evaluation covers representative RadioML 2018.01A schemes: 4ASK, BPSK, QPSK, 16PSK, 16QAM, FM, AM-DSB-WC, and 32APSK. This set covers analogue formats (FM, AM-DSB-WC) and digital schemes across amplitude, frequency, and phase modulation, enabling direct comparison with O’Shea et al.’s raw I/Q results on the same benchmark.

Fragmented architectural comparisons. The surveyed papers compare architectures, but do not isolate the successive effects of a baseline CNN, regularisation, augmentation, and transfer learning using the same representation and split. Contemporary attention-based architectures continue this fragmentation: Dinesh et al. (2025) report a 22.75-27.22% accuracy gain over baseline deep-learning methods on HISARMOD2019.1 using a multi-scale dual-attention network with wavelet feature fusion, but, like the other surveyed studies, evaluate only on benchmark data rather than consumer hardware. Following Chollet and Watson’s (2026) incremental workflow keeps each design choice connected to a measured change in performance.

Design (1974/2000 words)

Template Used

Template 3.2, Deep Learning Breast Cancer Detection, closely matches this project’s goals. It defines a deep-learning image-classification problem solved via CNNs using the Chollet and Watson (2026) workflow, which serves as my core methodology. The key difference is that this project classifies STFT spectrograms of RF signals rather than X-ray mammograms. These techniques (dataset splitting, model building, regularisation, transfer learning) apply directly to RF modulation classification. The project number used in this report is **3.2**.

Project Domain and Users

I designed the project for non-cooperative RF spectrum sensing and Automatic Modulation Classification (AMC), where a receiver estimates modulation without transmitter metadata (Chennagiri, Sehgal and Ravinder, 2024; Huynh-The et al., 2021; Zheng et al., 2025). The

intended users are spectrum-monitoring engineers, autonomous-platform developers, and researchers. Their needs are timely decisions, actionable uncertainty, and reproducible evidence that the model learned modulation structure rather than simulated-data artefacts. Concretely, a spectrum-monitoring engineer scanning for unlicensed transmissions needs a fast, per-capture confidence score to prioritise manual follow-up; an autonomous platform operating under fading and Doppler needs a decision that degrades gracefully rather than failing silently; and a researcher extending this work needs class- and SNR-resolved metrics to judge whether a result generalises before deployment.

I designed the workflow as decision support rather than automatic label generation. The user supplies an unknown I/Q capture, the system frames and normalises it, produces representations, returns probabilities, and exposes errors through SNR curves and confusion matrices. Confidence is useful only when the input resembles training. The HackRF experiment is a domain test, not evidence about every broadcast or receiver.

This analysis led me to six design requirements. First, I require **non-cooperative input**: processing must use only observed I/Q samples and must not require symbol timing, protocol metadata, or transmitter labels at inference time. This motivates signal representations derived from I/Q (Abd-Elaziz, Abdalla and Elsayed, 2023; Huynh-The et al., 2021). Second, I require **condition-aware evaluation**: I report results by SNR and class, because noise, fading, frequency offset, timing error, and Doppler can change the observation without changing the modulation (Zheng et al., 2025). Third, I require **affordable computation**: the pipeline must run with a consumer SDR and limited memory, making a small custom CNN a meaningful design candidate.

Fourth, I require **actionable output**: probabilities, per-class precision and recall, confusion matrices, and SNR curves must accompany headline accuracy so users can judge when a decision is dependable. Fifth, I require **representation coverage**: the input must preserve both temporal-frequency evidence for analogue signals and geometric evidence for closely related digital schemes. This directly motivates my five-channel representation rather than an unexplained increase in depth. Finally, I require **honest deployment boundaries**: the system must expose the sim-to-real gap and must not present confidence as certainty when receiver or waveform conditions differ from training.

Each requirement maps to a testable feature: the benchmark split and metrics, five-channel CNN input, class- and SNR-specific evaluation, and identical preprocessing for HackRF inference. The contribution is a test of whether explicit geometry improves low-cost AMC and whether that improvement survives on consumer hardware.

Design Justification

Eight representative RadioML 2018.01A schemes were selected, as introduced above. **4ASK** (Four-level Amplitude Shift Keying) uses four amplitude levels; **BPSK** (Binary Phase Shift Keying) uses two phase states; **QPSK** (Quadrature PSK) uses four; **16PSK** uses sixteen equally spaced phase states; **16QAM** (Quadrature Amplitude Modulation) combines amplitude and phase in a 4×4 grid; **32APSK** (Amplitude and Phase Shift Keying) uses thirty-two points on two amplitude rings; **FM** (Frequency Modulation) varies carrier frequency; and **AM-DSB-WC** (Amplitude Modulation, Double Sideband with Carrier) varies carrier amplitude while retaining the carrier. These eight classes together cover analogue and digital signals across amplitude, frequency, and phase.

This design extends Waqas et al. (2023) as detailed in the Literature Review: evaluation on RadioML 2018.01A, a full-range baseline alongside the $\text{SNR} \geq -2$ dB final model, and live HackRF One testing that the reviewed literature does not report.

My design follows Chollet and Watson’s (2026) incremental workflow: establish a performance floor with a small CNN, inspect data-specific failure modes, then test regularisation and transfer

learning. This order distinguishes problems in the input from problems in model capacity and limits unnecessary computation.

Project Structure

The completed work runs as a five-stage design pipeline, from raw dataset frames through to a model tested on live hardware captures; the Implementation chapter reports it as seven executed stages, because modelling was carried out in three separate rounds. Appendix C (Figure C1) shows the system architecture: RadioML frames and normalised HackRF captures both enter the same STFT and histogram preprocessing, producing the five-channel tensor consumed by the CNN, whose output feeds either the synthetic evaluation metrics or the OTA majority-vote inference path.

The workplan was divided into dated subtasks and milestones, broken down to the level of each individual training run (the baseline CNN, each of the three InceptionResNet-V2 configurations, and the EfficientNetB0 run) so that progress and blockers were visible at the task level rather than only at the phase level. Appendix C (Figure C2) covers planning, data preparation, modelling, evaluation, HackRF testing, reporting, and the demonstration video from 15 June to 15 September 2026. Its seven milestones mark the approved plan, preliminary report, validated data pipeline, recorded baseline and transfer results, completed synthetic evaluation, sim-to-real result, and final submission package.

1. **Data ingestion.** I read the RadioML 2018.01A HDF5 file (O’Shea et al., 2018) using h5py, extracting I/Q frames from group X, one-hot labels from Y, and SNR values from Z. After selecting the eight target classes, the data is split into training, validation, and test sets, following Chollet and Watson’s (2026) protocol for fitting, tuning, and unbiased evaluation. The split is balanced across modulation and retained SNR groups, and the test set remains untouched during training and tuning.
2. **Preprocessing.** Each 1,024-sample complex I/Q frame converts to a five-channel 128×128 feature tensor. Only frames with $\text{SNR} \geq -2$ dB are retained, removing ambiguous observations (Waqas et al., 2023).

The channel names describe feature roles rather than display colours. The network receives five numeric channels, while the R/G/B notation is retained to identify the first three feature maps in figures:

- **Log-Magnitude (R).** The STFT splits I/Q into overlapping time windows and measures frequency content in each window. Its log-magnitude image shows where signal energy is located in time and frequency. The logarithm compresses the large dynamic range, preserving weak structure beside strong components. This channel is particularly useful for FM frequency movement and AM energy bands, but it averages away much of the phase geometry needed to separate digital schemes.
- **Inst. Frequency (G).** This is the change in unwrapped STFT phase across time. It exposes frequency sweeps in FM and phase transitions in PSK/QAM signals, adding temporal phase evidence that magnitude alone cannot provide. Percentile clipping limits the effect of unstable phase estimates in low-energy bins.
- **Spectral Flux (B).** This is the time derivative of log-magnitude energy. It highlights changing envelopes and symbol transitions, complementing the relatively static energy pattern in Log-Magnitude. Spectral Flux was retained after comparison with the rejected group-delay feature showed more structured, class-dependent evidence (Appendix F).
- **Constellation (C).** A two-dimensional histogram of the raw I/Q samples records their amplitude-and-phase geometry. Unlike the STFT channels, it directly shows the four QPSK clusters, the 16PSK ring, the 16QAM grid, and the two 32APSK rings. This channel was added because the baseline confusion matrix showed that time-frequency energy alone could not separate these digital classes.

- **Phase-Time (P).** A histogram of instantaneous phase against normalised sample time records both angle clusters and their temporal distribution. It reinforces the geometry in Constellation while retaining information about phase transitions, making it complementary rather than an identical copy of the I/Q histogram.

The first three channels describe temporal-frequency dynamics, while the last two encode geometry discarded by power-based spectrograms. The three-channel model reached 63.65% accuracy, while the revised design reached 93.77% and raised confused-cluster macro-F1 from 0.275 to 0.877. These numbers measure the combined revision; they do not separate the contribution of either added channel individually.

3. **Modelling.** I began with a small three-block CNN, then tested regularisation, transfer learning, and feature engineering. The models use softmax outputs and Adam, later experiments use focal loss. Early stopping, best-weight checkpointing, and learning-rate reduction limit overfitting. Transfer learning used a frozen ImageNet base followed by low-rate fine-tuning. Results from other representations are not directly comparable (Abd-Elaziz, Abdalla and Elsayed, 2023; Rehman, Abbas and Shehzad, 2025).
4. **Evaluation.** Each class is scored with accuracy, precision, recall, macro-F1, confusion matrices, and SNR curves on the held-out test set.
5. **Sim-to-real transfer.** The best-performing model evaluated live signals captured via HackRF One (Great Scott Gadgets, 2026) using GNU Radio (GNU Radio Foundation, 2026). This assessed reliability on consumer hardware, where DC offset (Clark and Clark, 2025) alongside I/Q imbalance, oscillator drift, phase noise, and ADC non-linearity can degrade classification accuracy (Zheng et al., 2025).

Evaluation Plan

The evaluation plan follows Chollet and Watson’s (2026) held-out test-set protocol, extended here in project-specific ways stated before any result is shown. Two regimes are reported: an all-SNR baseline and a separate $\text{SNR} \geq -2$ dB final-model result, because the baseline confusion matrix showed that aggregate accuracy alone hides an attractor-class failure mode; every synthetic result is therefore reported per class and per SNR band alongside precision, recall, macro-F1, and the confusion matrix, not accuracy alone.

The single OTA test was a deliberate scope decision, not an oversight. The four transfer-learning experiments consumed most of the available time budget, with individual runs taking up to roughly 43 hours on CPU-only hardware. Given that constraint, I ran one OTA test on FM, the most representative analogue class, reasoning that a confident failure on the easiest class is strong evidence the sim-to-real gap is systemic rather than class-specific, which makes further single-class OTA tests a lower priority than completing the synthetic evaluation properly within the time actually available. Table 3a in the Evaluation chapter fixes pass/fail thresholds in advance, so the model is judged against a pre-declared bar, not just headline accuracy after the fact.

Key Technologies and Methods

NumPy, SciPy, and h5py implement I/Q framing, STFT processing, and HDF5 access; Keras runs the models on JAX (Chollet and Watson, 2026; O’Shea et al., 2018), with dropout, checkpointing, and learning-rate reduction as overfitting safeguards.

Design Evolution

The original design planned single-channel 64×64 log-magnitude spectrograms with transfer learning from an ImageNet-pretrained InceptionResNet-V2 backbone. Implementation sur-

faced three findings that required revision.

First, the single-channel 64×64 input provided little geometric information for distinguishing QPSK, 16PSK, 16QAM, and 32APSK. Their key differences lie in constellation geometry rather than time-frequency energy. The observed plateau suggested that changing the loss function would be less promising than adding a representation of this geometry.

Second, ImageNet features did not transfer cleanly to STFT spectrograms. Four documented transfer-learning runs, three using InceptionResNet-V2 (55.9 million parameters) and one using EfficientNetB0 (4.0 million parameters), all produced lower test accuracy than the simple three-block baseline, despite the pretrained backbones having significantly more capacity. The fundamental statistics of natural photographs are too different from spectrogram images for the learned filters to transfer usefully with this dataset size.

Third, CPU-only hardware made each transfer learning run a 12–43 hour commitment, reducing the scope for iterative exploration to a single committed configuration per run.

The final design therefore reverted to a custom CNN trained from scratch, extended the spectrogram from one to five channels by adding a 2D I/Q constellation histogram and a phase-time histogram that encode the geometric information the three-channel STFT lacked, and filtered the final-model training and test data to $\text{SNR} \geq -2$ dB. I made this filtering decision after the all-SNR baseline showed that the lowest-SNR samples were dominated by noise and that the digital classes remained confused even at high SNR with the original representation. Appendix F records the visual evidence behind these representation decisions. The final model was therefore evaluated in a defined higher-quality SNR regime, not across the full RadioML range.

Implementation (2431/2500 words)

The prototype is a runnable Jupyter notebook whose saved state implements the final five-channel pipeline end to end, from ingestion through held-out evaluation; the baseline and transfer-learning experiments reported below were run from successive versions of the same notebook, and the over-the-air stage runs from standalone scripts. The notebook began as the Week 2 eight-class learning milestone and was extended with transfer learning and feature engineering. Beyond standard coursework CNN training, the implementation required diagnosing a subtle batch-normalisation configuration error in a 55.9-million-parameter pretrained backbone, tuning layer-freeze boundaries across four transfer-learning configurations, engineering a five-channel spectrogram-plus-histogram representation from raw complex samples, and managing training under CPU-only, memory-constrained hardware. The HackRF inference boundary is implemented in the standalone `infer_ota.py` script, which rebuilds the final CNN and preprocessing pipeline for real captures. The notebook and supporting scripts are listed in Appendix B. Table 2 maps the notebook’s numbered steps onto the seven implementation stages and identifies where each stage is realised.

Table 2. Notebook-to-report pipeline mapping.

Notebook step	Pipeline stage	Subsection
Steps 1-1b	Environment preparation	Stage 1 (setup)
Steps 2-3	Data ingestion	Stage 1
Steps 4-5	STFT preprocessing	Stage 2
Steps 6-8, earlier version	Baseline CNN and training	Stage 3
Steps 9-10	SNR balanced evaluation	Stage 4
Earlier notebook versions	Transfer-learning experiments	Stage 5
Steps 4-8, saved version	Five-channel representation and final model	Stage 6

Notebook step	Pipeline stage	Subsection
Standalone scripts (Appendix B)	Capture normalisation and OTA inference	Stage 7

Data ingestion and class selection (Stage 1)

Environment and configuration (Step 1) Steps 1 and 1b set hyperparameters and random seeds for reproducibility. A helper `resolve_dataset_path` function downloads RadioML 2018.01A via kagglehub or uses a local override. The notebook reads HDF5 groups X (I/Q frames), Y (labels), and Z (SNR) per O’Shea et al. (2018).

Load RadioML 2018.01A and select the eight classes (Step 2) Step 2 selects eight of the 24 classes, reducing low-SNR ambiguity reported for similar modulation families (Abd-Elaziz, Abdalla and Elsayed, 2023; Nasir, Sheikh and Malik, 2025). Samples per cell are limited to keep training within the available time budget.

Balanced training, validation, and test sets (Step 3) Step 3 first retains frames at SNR ≥ -2 dB for the later feature-engineering and final-model experiments, then builds a three-way split balanced by (modulation, SNR) pairs. This separation of fit, tune, and final-check data follows Chollet and Watson’s (2026) protocol and enables SNR-oriented evaluation within the retained range. The earlier baseline used the corresponding all-SNR data before this filter was introduced.

Result (Stage 1) For the final feature-engineering sequence, I loaded 41,600 frames: 200 per (modulation, SNR) cell across eight classes and 26 SNR levels. The pre-declared SNR ≥ -2 dB filter retained 27,200 frames across 17 levels, which I divided into balanced training, validation, and test sets. I excluded lower-SNR frames rather than treating them as final-model test data. The 49.47% result is a separate all-SNR baseline and is not directly comparable without accounting for this change.

STFT spectrogram preprocessing (Stage 2)

STFT spectrogram transform (Step 4) Step 4 converts each 1024-sample complex I/Q frame into a log-magnitude STFT image, following Waqas et al. (2023). The logarithm compresses the wide energy range so strong components do not obscure weaker details.

Visual sanity check (Step 5) Step 5 applies a Hann window, balancing time and frequency resolution with a preference for timing clarity per Zheng et al. (2025). Signal strength is then converted to decibels, zero frequency is centred for readability (Clark and Clark, 2025), and min-max scaling is applied individually to each image to ensure consistent contrast across samples.

The initial prototype used 64×64 single-channel images. Waqas et al. (2023) demonstrated that resolution reduction can cut computational cost while preserving useful structure. The final model increased the STFT representation to 128×128 and added two geometry channels. One high-quality example per signal type is plotted using viridis. The network processes numerical feature channels rather than display colours.

Result (Stage 2) The initial transform produced 64×64 single-channel images. Analogue and low-order schemes showed structure, but the digital classes remained similar, leading to the later five-channel design. This single visual check anticipated the quantitative confusion later confirmed in Stage 4: a representation that only encodes time-frequency energy cannot separate classes that differ mainly in constellation geometry, regardless of how much training data or model capacity is applied.

Baseline CNN and training (Stage 3)

Build the baseline CNN (Step 6) Step 6 follows Chollet and Watson’s (2026) incremental workflow by starting deliberately small, designing a minimal three-block convolutional network (171,336 trainable parameters) with a dense head sized only to establish an above-chance baseline before scaling or regularisation. The baseline uses a softmax output, Adam optimiser, and categorical cross-entropy loss, with accuracy and macro-F1 used to monitor performance (Chollet and Watson, 2026).

Train with callbacks (Step 7) Step 7 uses early stopping with best-weight restoration, best-only checkpointing, and learning-rate reduction on plateau (Chollet and Watson, 2026). Only training and validation splits are used. The test set remains untouched until evaluation.

Training curves (Step 8) Step 8 plots loss and accuracy curves to identify convergence and overfitting. The residual-network evidence motivating the later transfer-learning experiments is discussed in the Design chapter.

Result (Stage 3) Training ran 17 epochs before early stopping. Validation loss reached minimum 1.11 at epoch 9 then rose while training accuracy climbed to 62.1% by the final epoch, signalling overfitting (Chollet and Watson, 2026). Because callbacks monitor validation loss, `restore_best_weights` reinstated the epoch 9 model weights (lowest-loss weights, validation accuracy 49.47%, training accuracy 53.68%) where training accuracy was 4.2% higher than validation accuracy. But by the final epoch, training accuracy is 12.1% higher than validation accuracy, showing the model is fitting the training set more and generalising less.

SNR balanced evaluation (Stage 4)

Evaluate against the random floor and confusion matrix (Step 9) Step 9 evaluates test accuracy against the 12.5% threshold to verify above-chance performance, performed only on the held-out test set (Chollet and Watson, 2026). A confusion matrix then reveals the hardest classification pairs. Published analysis predicts confusion among same-family digital schemes at low SNR (Abd-Elaziz, Abdalla and Elsayed, 2023; Nasir, Sheikh and Malik, 2025).

Result (Stage 4, Step 9) The baseline model achieved 49.47% test accuracy after restoring the best checkpoint from epoch 9, a 4.2 percentage-point drop from its training accuracy. Against the 12.5% random-guess floor, this represents roughly four times the chance accuracy and confirms the Week 2 above-chance-performance milestone (Chollet and Watson, 2026). However, the aggregated score masks significant issues revealed by the confusion matrix and per-class breakdown.

Analogue and low-order schemes are well-learned, with F1 scores: FM 0.8, 4ASK 0.78, AM-DSB-WC 0.77, BPSK 0.63. In contrast, four digital schemes collapse together: 16QAM 0.33, 32APSK 0.23, QPSK 0.20, 16PSK 0.02. The worst performer is 16PSK with precision 0.28 and recall 0.01. The confusion matrix shows 16QAM is acting as an attractor class that absorbs other digital schemes: 16PSK was predicted as 16QAM 375 times, 32APSK 373 times, and QPSK 354 times.

Accuracy versus SNR (Step 10) Step 10 plots accuracy against SNR levels, a standard presentation in the surveyed literature (Abd-Elaziz, Abdalla and Elsayed, 2023; Waqas et al., 2023; O'Shea et al., 2018). The curve is interpreted alongside per-class precision, recall, F1, and confusion matrices. Receiver operating characteristic curves were not used because the comparison is multiclass classification by class and SNR.

Result (Stage 4, Step 10) At extremely high noise levels around -20 dB, the model performs no better than random guessing. However, as signal quality improves, accuracy climbs steeply from 24% at -10 dB to 63% at -2 dB. Beyond this point, performance stabilises, plateauing between 60% and 67% all the way up to +30 dB. Specifically, the model reaches its peak accuracy of 66.7% at an SNR of 24 dB. On average, the model maintains a 63.5% accuracy in clean conditions (SNR > 10 dB), whereas it drops significantly to just 32.5% and below when noise dominates the signal (SNR < -2 dB).

The plateau at 63–67% even at the highest SNR levels is a critical diagnostic. When noise is negligible, any remaining confusion must come from the input representation itself: the single-channel log-magnitude spectrogram simply does not encode the geometric differences between QPSK, 16PSK, 16QAM, and 32APSK. This finding directed the next phase of the project.

Transfer learning experiments (Stage 5)

Following the design's planned next step, I attempted to improve on the 49.47% baseline by using transfer learning from an ImageNet-pretrained backbone. I ran four documented configurations across two architectures: InceptionResNet-V2 (three runs) and EfficientNetB0 (one run), with each run diagnosing the previous run's shortfall and testing a specific, adjusted hypothesis rather than repeating the same configuration. Each configuration followed Chollet and Watson's (2026) two-phase approach: first training a new classification head with the backbone frozen, then unfreezing upper layers for fine-tuning at a reduced learning rate.

InceptionResNet-V2, run v1. Phase 2 unfroze 680 of 780 backbone layers at learning rate 1×10^{-4} . This was the largest unfreeze in any run. Phase 1 validation accuracy flatlined at 34–37% across ten epochs. The base model was missing the `training=False` flag, so Keras ran batch-normalisation layers in training mode. This corrupted ImageNet statistics with spectrogram data. Phase 2 training accuracy climbed to 50% while validation stayed at 38%. Too many layers were unfrozen for the available data. Test accuracy: 38.00%.

InceptionResNet-V2, runs v2 and v3. Having identified an oversized unfreeze boundary as the likely cause, these runs tested a much smaller boundary. Correcting the layer-unfreeze boundary to 50 trainable layers avoided catastrophic forgetting but caused underfitting. Run v3 doubled the dataset and corrected the learning rate to 1×10^{-5} , producing a near-zero train-validation gap (40.3% vs 39.1%). Test accuracy: 38.97%.

EfficientNetB0. Since neither unfreeze boundary resolved the InceptionResNet-V2 shortfall, the next hypothesis targeted the backbone itself. Switching to a smaller backbone (4.0M vs 55.9M parameters) better matched the dataset size. Phase 1 rose from 31.1% to 37.7% validation accuracy over 20 epochs. Phase 2 reached 44.78% test accuracy. It was still improving at the 60-epoch ceiling. Training time: approximately 43 hours on CPU.

Result (Stage 5) All four configurations finished below the 49.47% baseline, the best of them EfficientNetB0 at 44.78%; Table 5 in the Evaluation chapter lists each run's test accuracy. The baseline trained in approximately 6.5 minutes, whereas transfer-learning runs required 12–43 hours. This limited iterative tuning on the CPU-only machine.

Feature engineering and the final model (Stage 6)

In sequence, the project tested a progression of model configurations: a 3-block CNN trained from scratch on a single-channel spectrogram (the baseline); the same 3-block architecture retrained on three independent feature channels after the SNR filter (the intermediate model); four transfer-learning configurations that replaced the 3-block CNN with a pretrained ImageNet backbone (InceptionResNet-V2 or EfficientNetB0); and the final four-block residual CNN trained on the five-channel input. Each configuration changed at least one of architecture, input representation, resolution, or pretrained weights relative to its predecessor; Table 5 and the Design Evolution subsection identify which factors changed at each step and flag where multiple factors changed together, so a reader can see exactly where a clean attribution is and is not possible.

The four documented transfer-learning runs suggested that the tested pretrained configurations were not an effective foundation under the available dataset, training budget, and pre-processing choices. One run also contained a batch-normalisation configuration error, and EfficientNetB0 was still improving when its epoch limit was reached. Following Chollet and Watson’s (2026) recommendation to diagnose the data before scaling the model, I reverted to a custom CNN and targeted the input representation directly.

SNR filter. Frames below -2 dB SNR were removed from training. At very low SNR, constellation points become difficult to distinguish: the four-point QPSK cluster, the sixteen-point 16PSK ring, and the 4×4 16QAM grid can be obscured by noise. These frames were difficult for the tested representations and added noisier examples to training. The filter reduced training samples from 41,600 to 27,200 but substantially improved the quality of training signal. Waqas et al. (2023) applied a similar restriction ($\text{SNR} \geq 5$ dB) in their 91.2%-accuracy result.

Three-channel intermediate model. As a first step, I replaced the single-channel log-magnitude spectrogram with three independent feature maps: log-magnitude (R), instantaneous frequency (G), and spectral flux (B). Combined with the SNR filter, this raised test accuracy to 63.65%. Analogue classes reached near-perfect F1 (FM 1.000, 4ASK 1.000, AM-DSB-WC 0.998) and BPSK improved dramatically from 0.063 to 0.980. However, the four digital PSK/QAM classes remained confused, rotating misclassifications among themselves regardless of whether I applied focal loss ($\gamma=2$), class weights, or cosine decay learning rate schedules. The best confused-cluster macro-F1 achievable with loss function tuning alone was 0.275.

Adding the constellation and phase-time channels. The stagnation confirmed the input representation was the ceiling. I added two channels that encode digital geometry. Channel 4 is a 2D density histogram of raw I vs Q values over the $[-2, +2]$ range. QPSK produces four clusters, 16PSK produces a ring, 16QAM produces a rectangular grid, and 32APSK produces two rings. These differences are invisible to the STFT but visible in the I/Q histogram. Channel 5 is a 2D histogram of instantaneous phase $\angle(I + jQ)$ over time.

The STFT resolution increased from 64×64 to 128×128 . The CNN was extended from three to four convolutional blocks with a residual connection. The dense head was widened to 256 neurons.

Result (Stage 6) The combined result was a step-change: test accuracy jumped from 63.65% to **93.77%**, confused-cluster macro-F1 rose from 0.275 to 0.877, and the largest single confusion (16QAM predicted as 32APSK) fell from 253 instances to 59. Total training time on the same CPU hardware: approximately 1.5 hours.

Over-the-air capture setup (Stage 7)

To test the trained model on real hardware, I used a HackRF One SDR (Great Scott Gadgets, 2026) with GNU Radio (GNU Radio Foundation, 2026) to capture a live FM broadcast.

The HackRF was tuned to a 96 MHz hardware centre frequency. A Xlating Finite Impulse Response (FIR) filter selected the 94.9 MHz station and decimated the stream to 400 kilosamples per second (kSps). The output is a binary I/Q file in the RadioML format. Figure B1 in Appendix B reproduces the GNU Radio flowgraph used for this capture. A parallel demodulator tap on the same stream produced intelligible station audio throughout, confirming the receiver was locked to the intended broadcast before any frame was written to disk.

RadioML 2018.01A normalises every 1,024-sample frame to unit variance. The HackRF capture has arbitrary power, so a normalisation script divided each frame by its standard deviation. A standalone inference script rebuilt the CNN, loaded the saved weights, and computed the majority vote.

Result (Stage 7) The documented majority-vote inference result used 2,725 frames, the complete content of the 22.3 MB capture file actually used for inference (2,790,718 raw samples, ~7 seconds at 400 kSps) rather than a truncated subsample. A separate project note, from an earlier successful capture run, records 24,000,000 samples normalising to 23,437 frames — consistent with a full ~60-second recording at 400 kSps — but that longer capture was a different session and was not the file carried through to the reported OTA result. I therefore report the 2,725-frame result, which is the full content of the file actually used for inference. Even restricted to 2,725 frames, the majority vote is computed over roughly 7 seconds of continuous baseband capture at 400 kSps, which is large enough that the result reflects a stable model decision rather than a single noisy frame.

Evaluation (2049/2500 words)

Results follow the experimental sequence described in the Design chapter. Each synthetic test split was separate from training and validation (Chollet and Watson, 2026). The all-SNR baseline used 41,600 frames split into 29,120 training, 6,240 validation, and 6,240 test frames. The final experiment used 27,200 frames split into 19,040, 4,080, and 4,080. Each split was stratified by class and SNR. The reported runs used seed 42 once, so uncertainty across seeds was not measured. These are successive experiments with different data regimes. I report accuracy, precision, recall, F1, macro-F1, and confusion matrices. Every training-history, confusion-matrix, per-class, and SNR figure below is exported directly from the notebook run of the experiment version it documents. Table 3a states the pre-declared pass/fail criteria and Table 3b records the outcome against each.

Table 3a. Evaluation criteria and test conditions.

Metric	Test data	Threshold
Above-chance classification	Synthetic test set	> 12.5% accuracy
Final synthetic accuracy	Test set, SNR ≥ -2 dB	$\geq 80\%$ across 8 classes
Per-class discrimination	Filtered test set	F1 ≥ 0.80 per class
OTA reliability	Labelled OTA capture	Majority vote matches label

Table 3b. Evaluation results and interpretations.

Metric	Observed result	Interpretation
Above-chance classification	49.47% baseline; 93.77% final	Pass: both exceed random chance.
Final synthetic accuracy	93.77%	Pass: meets the synthetic target.

Metric	Observed result	Interpretation
Per-class discrimination	F1 0.842–1.000 for all 8 classes	Pass: all classes retained.
OTA reliability	QPSK predicted for an FM broadcast	Fail: waveform-level domain gap, not receiver quality.

The OTA criterion is intentionally stricter than confidence: a high-confidence prediction is not reliable unless it matches the known modulation label. The synthetic thresholds are project acceptance criteria, not significance tests. They apply only to the filtered regime and do not imply performance across the full RadioML SNR range.

Baseline model evaluation

The three-block prototype CNN trained on all 26 SNR levels reached **49.47% test accuracy**, 3.96× the 12.5% random-guess floor. The 12.5% floor is the chance probability for eight equally likely classes. This is the historical all-SNR baseline, not the final filtered-model result. Table 4 reports its per-class F1 scores. Appendix D (Figures D1 and D2) shows the training history and confusion matrix.

Per-class F1 scores revealed a sharp divide between modulation families:

Table 4. Baseline per-class F1 scores on the full-SNR test set.

Class	F1	Notes
FM	0.80	Distinct frequency-sweep pattern
4ASK	0.78	Unique four-level amplitude envelope
AM-DSB-WC	0.77	Characteristic amplitude modulation
BPSK	0.63	Two-phase pattern recognisable at high SNR
16QAM	0.33	Attractor class: absorbs other digital predictions
32APSK	0.23	Frequently confused with 16QAM
QPSK	0.20	Frequently confused with 16QAM
16PSK	0.02	Precision 0.28, recall 0.01, essentially invisible

The confusion matrix revealed that 16QAM acted as an attractor class: 375 of 16PSK frames, 373 of 32APSK frames, and 354 of QPSK frames were all predicted as 16QAM. The SNR-resolved accuracy curve showed the model performing at random chance at −20 dB, climbing to 63% at −2 dB, and then plateauing between 60–67% at all SNR levels above 0 dB. The plateau at high SNR confirmed the problem was the input representation, not the signal quality.

Transfer learning evaluation

Four configurations across two pretrained backbones produced lower test accuracy than the simple baseline. Table 5 summarises these results. Appendix D (Figures D3 and D4) shows the EfficientNetB0 training history and confusion matrix.

Table 5. Transfer-learning comparison.

Configuration	Test accuracy	Attractor class	Primary failure
Baseline 3-block CNN	49.47%	16QAM	Mild overfit
InceptionResNet-V2 v1	38.00%	32APSK	Catastrophic forgetting

Configuration	Test accuracy	Attractor class	Primary failure
InceptionResNet-V2 v2	36.63%	16QAM + BPSK	Dual-attractor collapse
InceptionResNet-V2 v3	38.97%	16QAM	Underfitting
EfficientNetB0	44.78%	None (distributed)	Epoch-budget limited

EfficientNetB0 was the most effective transfer-learning run, showing no attractor collapse. It was still improving at the 60-epoch ceiling and remained 4.69 percentage points below the three-block baseline. The InceptionResNet-V2 runs produced attractor-class collapse in addition to the baseline’s distinct 16QAM attractor pattern.

The computational cost is itself an evaluation finding. On a CPU-only 16 GB laptop, the baseline completed in 6.5 minutes while EfficientNetB0 required 43 hours. This meant every transfer learning configuration was a single committed run with no budget for iterative tuning. The baseline, by contrast, could be varied and re-trained freely.

Three-channel intermediate model evaluation

Switching to three independent feature channels and applying the $\text{SNR} \geq -2$ dB filter raised test accuracy to **63.65%** ($5.09\times$ chance lift). Analogue classes reached near-perfect F1 (FM 1.000, 4ASK 1.000, AM-DSB-WC 0.998, BPSK 0.980). The confused digital cluster remained problematic. The intermediate channel comparisons are shown in Figures F1-F3 in Appendix F. Three interventions were tested:

- **Class weights** (upweighting confused digital classes): shifted the attractor from 32APSK to 16PSK without moving the confused-cluster macro-F1 above 0.240.
- **Focal loss, $\gamma=2$** : redistributed confusions more evenly across the four classes. Confused-cluster macro-F1 improved from 0.240 to 0.275. Overall accuracy was unchanged ($0.6365 \rightarrow 0.6347$) because the ceiling was in the input representation.
- **Cosine decay learning rate**: marginally regressed accuracy ($0.6365 \rightarrow 0.6187$), confirming no optimiser improvement was available.

All three experiments produced the same pattern: the four digital classes rotated errors among themselves. The tested STFT representation and training setup did not provide enough information to separate them reliably.

Final model evaluation

Adding the constellation and phase-time histograms within the revised 128×128 model produced a step-change result. Test accuracy reached **93.77%** ($7.50\times$ chance lift, Loss: 0.0573). Table 6 reports the final per-class metrics. Appendix D (Figures D5 and D6) shows the final training history and confusion matrix; Figure D7 gives the same metrics visually.

These figures measure the complete revised design, not a controlled ablation of the two added channels. Resolution, CNN capacity, channel count, and the SNR-filtered data regime all changed between the intermediate and final runs. Quantifying the independent effect of each variable — Constellation, Phase-Time, resolution, architecture — would require matched experiments outside the available compute budget.

Per-class results:

Table 6. Final five-channel CNN per-class precision, recall, and F1 scores.

Class	Precision	Recall	F1
4ASK	0.996	1.000	0.998
BPSK	0.996	0.994	0.995
QPSK	0.900	0.914	0.907
16PSK	0.923	0.875	0.898
16QAM	0.870	0.816	0.842
FM	1.000	1.000	1.000
AM-DSB-WC	1.000	1.000	1.000
32APSK	0.823	0.904	0.862

The remaining confusions are the hardest pairs: 16QAM predicted as 32APSK 59 times, 16PSK predicted as 32APSK 26 times, and 32APSK predicted as 16QAM 25 times. These pairs share constellation geometry (Abd-Elaziz, Abdalla and Elsayed, 2023; Nasir, Sheikh and Malik, 2025).

The SNR-resolved accuracy curve (Appendix D, Figure D8) showed a qualitatively different profile from the baseline: it starts at 64% at -2 dB, rises to 81% at 2 dB, 93% at 4 dB, and 95% at 6 dB, then reaches about 99% above 10 dB. This is consistent with the model using geometric information, but the single seed and non-ablated design prevent a stronger causal claim.

The 93.77% result is numerically close to Waqas et al.’s (2023) 91.2% result for STFT spectrograms, but the datasets, class counts, representations, and SNR conditions differ, so the figures are not directly comparable. O’Shea et al. (2018) reported approximately 95% at high SNR using raw I/Q across 24 classes, also under different conditions. The result here is achieved on $\text{SNR} \geq -2$ dB data. Full-range performance is lower.

Over-the-air experiment

Setup and result. The trained model was applied to a normalised live FM broadcast capture at 94.9 MHz. It classified the capture as **QPSK**: 2,183 of 2,725 frames (80.1%) voted QPSK with mean confidence of 91.9%. The expected answer was FM. The model was wrong and highly confident.

Figures E1–E3, shown in Appendix E, present three five-channel inputs sampled from this capture. Each figure has five panels: the first three preserve temporal-frequency structure, and the final two encode constellation density and phase over time. Each panel is a feature map rather than a colour photograph, presented there to make the representation and the nature of the failure inspectable. The panels are per-frame contrast-normalised, so they evidence signal structure and confirm that all five channels received in-range data; they are not a measure of absolute signal quality, which is quantified separately below.

Analysis. The misclassification reflects the domain gap between RadioML’s synthetic FM class and real broadcast FM. RadioML 2018.01A generates FM as narrow-band test tones with simulated random carrier offset and multipath (O’Shea et al., 2018). Real stereo broadcast FM is structurally richer than a single abstract FM waveform: it carries a mono audio sum (0–15 kHz), a 19 kHz stereo pilot tone, a 38 kHz double-sideband stereo difference channel, and a Radio Data System (RDS) subcarrier at 57 kHz. The stereo pilot appears as a persistent spectral line, the 38 kHz subcarrier creates a distinct energy band, and real programme audio produces statistics unlike the synthetic FM training examples. With no exposure to real broadcast characteristics, the model mapped the signal to the closest known pattern: QPSK.

O’Shea et al. (2018) reported the same direction: a model trained only on RadioML synthetic data lost approximately 10% accuracy on real OTA captures, and fine-tuning on a small number of labelled real-world examples partially recovered it. The project had a real FM capture, but no labelled real FM training examples were available for fine-tuning.

The receiver was not the limiting factor. The project’s stated question was whether classification survives on consumer-grade hardware, where phase noise, DC offset, I/Q imbalance, and ADC non-linearity were expected to be the risk. Those impairments were measured directly on the captured stream rather than inferred, using the `measure_capture.py` script listed in Appendix B, and each was small. DC offset sat 52.9 dB below signal power, helped by the deliberate -1.1 MHz offset tuning that places the receiver’s DC spike outside the selected channel. I/Q image rejection was 68.2 dB with a gain imbalance of 0.003 dB. No sample in the capture was clipped, leaving 10.0 dB of crest-factor headroom. Across all 2,725 frames the per-frame power varied by only 1.45 $\times$, with no dead-air or dropout frames. The demodulated signal was also well formed: the occupied bandwidth was 154 kHz at -20 dB, matching a standard FM broadcast channel, the multiplex baseband peaked 29.5 dB above the demodulated noise floor, and the 19 kHz stereo pilot resolved 15.8 dB above that floor, centred within one 12.2 Hz analysis bin of its nominal frequency. The prediction was correspondingly stable rather than erratic: 80.1% of 2,725 independent frames agreed on one class at 91.9% mean confidence, which is the signature of a consistent, well-formed input being mapped to the wrong label.

Two limits on this evidence should be stated. The figures characterise the decimated stream the model actually consumed, not the receiver front end in isolation, and a true in-band signal-to-noise ratio cannot be recovered from this file because the channel filter’s stopband begins at roughly 80 kHz and removes the out-of-band noise reference. The 38 kHz stereo difference band and the 57 kHz RDS subcarrier also sat within 0.4 dB of an empty guard band, so this capture confirms only the pilot, not the full multiplex. Taken together the measurements support the conclusion that a low-cost SDR was sufficient to receive and represent the signal, and that the failure sits in the training distribution: real broadcast FM carries structure, the 19 kHz pilot among it, that the synthetic FM class the model learned does not. This is a claim about the data rather than about the hardware, and it rests on converging evidence rather than an isolating experiment. Replaying a known synthetic RadioML waveform through a transmitter and recapturing it with the same HackRF would separate receiver impairment from waveform mismatch directly; that test was not run.

Planned extensions. This OTA subsection is a foundation for future experiments. Planned next steps include capturing AM-DSB-WC broadcasts (which can be verified by listening), and generating controlled digital transmissions (BPSK, QPSK) using a second SDR as a software transmitter to test whether the model’s near-perfect synthetic performance transfers to real digital signals where the domain gap may be narrower.

Critical assessment. The OTA failure does not invalidate the 93.77% synthetic result. It confirms that the sim-to-real gap remains unsolved (O’Shea et al., 2018). The capture chain and the inference pipeline both completed correctly on consumer hardware; what limited reliability was the training distribution. Closing this gap requires labelled real examples or broader domain variation. Judged against the project’s own stated objective of testing whether spectrogram-based classification generalises from simulation to consumer hardware, the result is genuinely mixed and splits cleanly in two: the hardware half of the question was answered positively, because the low-cost receiver captured a signal the pipeline could process, and the synthetic objective was met with a wide margin, but the sim-to-real objective was not, and both findings must be reported together rather than letting the stronger result overshadow the weaker one.

Conclusion (900/1000 words)

This project built a CNN that classifies radio modulation types from spectrogram images and tested it on a live signal captured with a consumer-grade HackRF One SDR. The central question was whether the approach holds up in real-world conditions. The result was strong performance on the filtered synthetic split, but a substantial limitation on the single OTA test.

What was achieved. Starting from a minimal three-block CNN reaching 49.47% accuracy

on the full SNR range, with 16QAM acting as an attractor class that absorbed other digital predictions, I arrived at a five-channel 128×128 custom CNN achieving **93.77% test accuracy** on eight modulation classes at $\text{SNR} \geq -2$ dB. For context, Waqas et al. (2023) report 91.2% on an 11-class STFT spectrogram problem and O’Shea et al. (2018) approximately 95% on raw I/Q sequences across 24 classes, but those studies use different datasets, class counts, and SNR ranges, so the figures sit alongside this result rather than being beaten by it. At $\text{SNR} > 10$ dB, the final model achieves 99% accuracy on its filtered synthetic test set. I did not obtain a full-range test result for the final representation, so the 93.77% figure must not be interpreted as full-range performance.

Main finding. Four tested transfer-learning configurations using InceptionResNet-V2 and EfficientNetB0 underperformed the 171K-parameter three-block baseline, although the runs used different configurations and budgets. The combined revision improved confused-cluster macro-F1 from 0.275 to 0.877 and achieved 93.77% accuracy on the filtered SNR regime. Because the final experiment also changed resolution, capacity, and data regime, no single variable can be credited: the result points toward representation engineering as a productive direction without quantifying which change drove most of the gain.

Hardware constraints as a project shaping force. CPU-only training with 16 GB RAM made transfer-learning runs take hours or days, while the custom CNN could be re-trained quickly. This limited hyperparameter exploration but also showed that the smaller model was better matched to this dataset. Inference latency and power consumption were not measured.

The OTA experiment and the domain gap. The HackRF One captured a live FM broadcast at 94.9 MHz. The model classified it as QPSK with 91.9% mean confidence. Real stereo broadcast FM contains a 19 kHz pilot, a 38 kHz difference channel, and an RDS subcarrier. These features differ from the synthetic FM examples used for training. The consumer radio itself was not the weak link, and this was measured rather than assumed: DC offset sat 52.9 dB below signal power, I/Q image rejection was 68.2 dB, no sample was clipped, all 2,725 frames were valid with power varying by only 1.45×, and the demodulated multiplex peaked 29.5 dB above the noise floor with the 19 kHz stereo pilot resolved 15.8 dB above it. The evidence therefore points at the signal rather than the receiver. A low-cost SDR was more than enough to receive and represent the broadcast, while the synthetic FM class the model learned was not enough to describe it. One capture and no loopback replay test still stop this being a conclusive isolation of the two causes. This mirrors the domain gap O’Shea et al. (2018) reported when transferring a synthetic RadioML model to OTA data, and it reinforces Zheng et al.’s (2025) identification of hardware and deployment variability as an open challenge rather than a problem this project alone could resolve.

Limitations. The 93.77% result is on the $\text{SNR} \geq -2$ dB filtered subset. The all-SNR curve belongs to the earlier single-channel baseline. Only 8 of RadioML 2018.01A’s 24 classes were evaluated. OTA testing used one FM capture. The project notes contain two frame counts because an earlier ~60-second capture run (23,437 frames) was a separate session from the shorter ~7-second capture (2,725 frames) actually used for the reported inference; only the shorter file has matching predictions. OTA experiments for AM-DSB-WC and digital signals were not completed. The final run used one seed and one OTA capture.

Balanced verdict. Taken together, the project met its synthetic accuracy objective convincingly but did not meet its sim-to-real reliability objective, and the reason was the training data rather than the price of the radio: a single OTA test is not enough to generalise from, and the pipeline, while reproducible and accurate on held-out synthetic data, has not been shown to work reliably outside the conditions it was trained under. Neither the 93.77% synthetic result nor the confident OTA misclassification should be read in isolation from the other.

Future work. The main next steps are fine-tuning on labelled real examples, completing AM-DSB-WC and digital OTA experiments, replaying a known synthetic waveform through the same HackRF to isolate receiver impairment from waveform mismatch, extending Efficient-

NetB0 training on GPU hardware, and evaluating all 24 RadioML classes.

Broader significance. The project provides evidence that explicit constellation geometry can matter more than architectural complexity for this filtered benchmark, extending the image-based representations that Huynh-The et al. (2021) catalogued alongside raw I/Q and constellation approaches. Within the literature reviewed for this project, no prior study combined a five-channel geometric-plus-spectral representation with a consumer-hardware OTA test on the same standardised benchmark; this combination, rather than either element alone, is the project's most original contribution, even though the OTA result itself was negative. The practical contribution is a reproducible five-channel CNN pipeline together with a direct demonstration that synthetic accuracy does not guarantee reliability on real signals, even when the consumer receiver capturing them performs adequately.

Appendices

Appendix A - Bibliography

Abd-Elaziz, O.F., Abdalla, M. and Elsayed, R.A. (2023) Deep learning-based automatic modulation classification using robust CNN architecture for cognitive radio networks. *Sensors*, 23(23), 9467.

Chennagiri, R., Sehgal, S. and Ravinder, Y. (2024) A survey on automatic modulation classification techniques. In *2024 Intelligent Systems and Machine Learning Conference (ISML)*, pp. 95–100. DOI: 10.1109/ISML60050.2024.11007395.

Chollet, F. and Watson, M. (2026) *Deep Learning with Python*. 3rd ed. Shelter Island: Manning Publications Co.

Clark, D. and Clark, P. (2025) *Practical SDR: Getting Started with Software-Defined Radio*. San Francisco: No Starch Press.

Dinesh, G. et al. (2025) Automatic modulation classification scheme for next-generation cellular networks using optimized adaptive multi-scale dual attention network. *Peer-to-Peer Networking and Applications*. DOI: 10.1007/s12083-025-01959-0.

GNU Radio Foundation (2026) GNU Radio: Free and open-source software for software-defined radios [Online]. Available from: <https://www.gnuradio.org/> [24 June 2026].

Great Scott Gadgets (2026) HackRF One: Open source hardware software-defined radio [Online]. Available from: <https://greatscottgadgets.com/hackrf/> [24 June 2026].

Huynh-The, T. et al. (2021) Automatic modulation classification: A deep architecture survey. *IEEE Access*, 9, pp. 142950–142971. DOI: 10.1109/ACCESS.2021.3120419.

Nasir, S., Sheikh, S.A. and Malik, F.M. (2025) Automatic modulation classification using convolutional neural network and support vector machine. *Digital Signal Processing*, 164, 105249. DOI: 10.1016/j.dsp.2025.105249.

O'Shea, T.J., Roy, T. and Clancy, T.C. (2018) Over-the-air deep learning based radio signal classification. *IEEE Journal of Selected Topics in Signal Processing*, 12(1), pp. 168–179. DOI: 10.1109/JSTSP.2018.2797022.

Rehman, F.U., Abbas, Q. and Shehzad, M.K. (2025) DL-AMC: Deep learning for automatic modulation classification [Preprint]. arXiv:2504.08011 [eess.SP]. Available from: <https://arxiv.org/abs/2504.08011> [19 August 2026].

Samarkandi, A. et al. (2023) Combined signal representations for modulation classification using deep learning: Ambiguity function, constellation diagram, and eye diagram. In *2023 32nd Wireless and Optical Communications Conference (WOCC)*. DOI: 10.1109/WOCC58016.2023.10139474.

Waqas, M., Ashraf, M. and Zakwan, M. (2023) Modulation classification through deep learning using resolution transformed spectrograms [Preprint]. arXiv:2306.04655 [eess.SP]. Available from: <https://arxiv.org/abs/2306.04655> [19 August 2026].

Zheng, Q. et al. (2025) Recent advances in automatic modulation classification technology: Methods, results, and prospects. *International Journal of Intelligent Systems*, 2025, Article ID 4067323. DOI: 10.1155/int/4067323.

Appendix B - Reproducibility and source code

The complete project, including the notebook, the over-the-air scripts, and this report, is available in the public repository at <https://github.com/z320817/CM3070>.

The main implementation is provided in the final.ipynb notebook. Its cells follow the report pipeline: environment and dataset setup, class selection and balanced splitting, five-channel preprocessing, CNN construction and training, held-out evaluation, and SNR analysis. The notebook is the primary source for the reported synthetic-data results and contains the plots and text summaries used during evaluation.

The OTA preprocessing and inference are provided separately so that the hardware experiment can be reproduced without rerunning training:

Table 7. OTA reproducibility artifacts.

Artifact	Purpose
load-and-normalize.py	Converts GNU Radio complex64 capture to unit-variance 1,024-sample I/Q frames.
measure_capture.py	Measures DC offset, I/Q imbalance, clipping, frame stability, occupied bandwidth, and the demodulated multiplex baseband.
infer_ota.py	Loads saved CNN weights, predicts each frame, returns majority-vote result.
OTA experiment report	Documents HackRF setup, classification result, and domain-gap analysis.

The capture itself was produced by the GNU Radio flowgraph shown in Figure B1: a Soapy HackRF Source (8 megasamples per second, 96 MHz centre) feeds a Frequency Xlating FIR Filter (decimation 20, -1.1 MHz offset for the 94.9 MHz station) whose 400 kSps output is written directly to the File Sink (fm_c96.3M_s400k.iq). A parallel wideband FM receive $\rightarrow$ Rational Resampler $\rightarrow$ Audio Sink chain, alongside a graphical frequency sink, let me monitor the demodulated audio and spectrum live to confirm the correct station was tuned before and during each capture; only the File Sink tap feeds the model pipeline.

Figure B1. GNU Radio flowgraph used to capture the 94.9 MHz FM broadcast: the Frequency Xlating FIR Filter branch feeds the File Sink used for model inference, while the wideband FM receive/Audio Sink branch was used only for live monitoring during capture.

The scripts intentionally duplicate the notebook’s preprocessing and model definition at the inference boundary. This reduces the risk that the OTA test uses a different representation from the one used during training. The report’s implementation and evaluation claims should therefore be read alongside these artifacts, rather than as unsupported pseudocode.

The following figures include the complete contents of the two executable OTA scripts as readable, line-numbered source-code pages. The original files remain the authoritative, runnable versions.

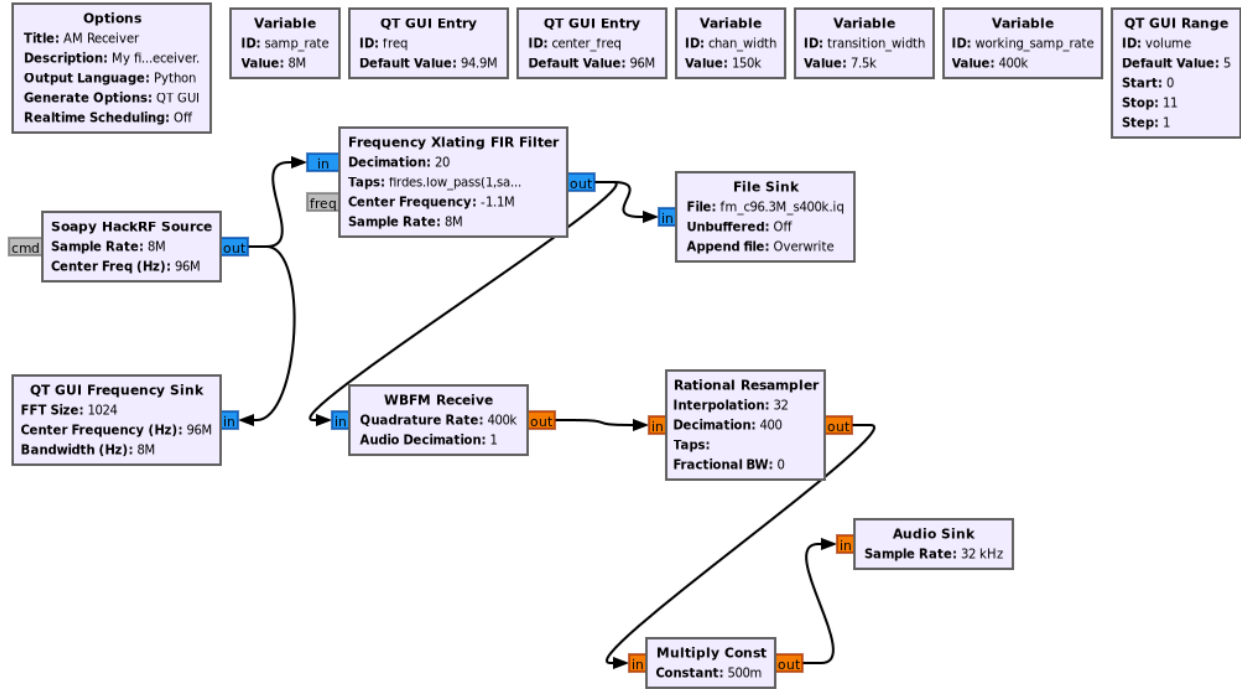


Figure 1: GNU Radio flowgraph used for the HackRF FM capture

Figure B2. *infer_ota.py*, source page 1.

Figure B3. *infer_ota.py*, source page 2.

Figure B4. *infer_ota.py*, source page 3.

Figure B5. *infer_ota.py*, source page 4.

Figure B6. *load-and-normalize.py*, source page 1.

Figure B7. *load-and-normalize.py*, source page 2.

The main notebook implementation is reproduced below as numbered source-code pages. Notebook code cells are marked in the extracted source so the implementation remains traceable to *final.ipynb*.

Figure B8. *final.ipynb* executable source, page 1.

Figure B9. *final.ipynb* executable source, page 2.

Figure B10. *final.ipynb* executable source, page 3.

Figure B11. *final.ipynb* executable source, page 4.

Figure B12. *final.ipynb* executable source, page 5.

Figure B13. *final.ipynb* executable source, page 6.

Figure B14. *final.ipynb* executable source, page 7.

Figure B15. *final.ipynb* executable source, page 8.

Figure B16. *final.ipynb* executable source, page 9.

Figure B17. *final.ipynb* executable source, page 10.

Figure B18. *final.ipynb* executable source, page 11.

```

1 #!/usr/bin/env python3
2 # Classify a normalised OTA IQ capture using saved baseline_cnn weights.
3 # Usage: python infer_ota.py <normalized.iq> <path/to/baseline_cnn.weights.h5>
4 # Example: python infer_ota.py fm_c96.3M_s400k_normalized.iq ../../baseline_cnn.weights.h5
5
6 import os
7 import sys
8 import math
9 from collections import Counter
10
11 import numpy as np
12 import matplotlib
13 matplotlib.use("Agg") # non-interactive backend – no display required
14 import matplotlib.pyplot as plt
15 from scipy.signal import stft as scipy_stft
16
17 # Must be set before keras is imported
18 os.environ.setdefault("KERAS_BACKEND", "jax")
19
20 import keras
21 from keras import layers
22
23 # — Constants – must match the notebook exactly —————
24
25 IMG_SIZE = 128
26 BATCH_SIZE = 32
27 FRAME_LEN = 1024
28 CLASSES = ['4ASK', 'BPSK', 'QPSK', '16PSK', '16QAM', 'FM', 'AM-DSB-WC', '32APSK']
29
30 # — Architecture – identical to the notebook —————
31
32 def residual_block(x, filters):
33     shortcut = layers.Conv2D(filters, 1, padding='same')(x)
34     x = layers.Conv2D(filters, 3, padding='same', activation='relu')(x)
35     x = layers.Conv2D(filters, 3, padding='same')(x)
36     return layers.Activation('relu')(layers.Add()([x, shortcut]))
37
38 def build_baseline_cnn(input_shape, n_classes):
39     inputs = keras.Input(shape=input_shape)
40     x = layers.Conv2D(16, 3, activation='relu')(inputs)
41     x = layers.MaxPooling2D()(x)
42     x = layers.Conv2D(32, 3, activation='relu')(x)
43     x = layers.MaxPooling2D()(x)
44     x = layers.Conv2D(64, 3, activation='relu')(x)
45     x = layers.MaxPooling2D()(x)

```

Figure 2: Inference script, source page 1

```

1     x = residual_block(x, 64)
2     x = layers.Conv2D(128, 3, activation='relu')(x)
3     x = layers.MaxPooling2D()(x)
4     x = layers.Flatten()(x)
5     x = layers.Dropout(0.1)(x)
6     x = layers.Dense(256, activation='relu')(x)
7     outputs = layers.Dense(n_classes, activation='softmax')(x)
8     return keras.Model(inputs, outputs, name="baseline_cnn")
9
10 # — Spectrogram pipeline – identical to the notebook —————
11
12 def to_spectrogram(iq, nperseg=128, noverlap=112):
13     x = iq[:, 0] + 1j * iq[:, 1]
14     _, _, Z = scipy.stft(x, window='hann', nperseg=nperseg, noverlap=noverlap,
15                          return_onesided=False, boundary=None, padded=False)
16     Z_shifted = np.fft.fftshift(Z, axes=0)
17     S = np.abs(Z_shifted)
18
19     R_raw = 20.0 * np.log10(S + 1e-12)
20     R = (R_raw - R_raw.min()) / (R_raw.max() - R_raw.min() + 1e-12)
21
22     phi_t = np.unwrap(np.angle(Z_shifted), axis=1)
23     d_phi_t = np.diff(phi_t, axis=1)
24     IF = np.concatenate([d_phi_t[:, :1], d_phi_t], axis=1)
25     if_lo, if_hi = np.percentile(IF, 1), np.percentile(IF, 99)
26     IF = np.clip(IF, if_lo, if_hi)
27     IF = (IF - if_lo) / (if_hi - if_lo + 1e-12)
28
29     d_R = np.diff(R_raw, axis=1)
30     SF = np.concatenate([d_R[:, :1], d_R], axis=1)
31     sf_lo, sf_hi = np.percentile(SF, 1), np.percentile(SF, 99)
32     SF = np.clip(SF, sf_lo, sf_hi)
33     SF = (SF - sf_lo) / (sf_hi - sf_lo + 1e-12)
34
35     def _rsz(arr):
36         t = arr[np.newaxis, :, :, np.newaxis]
37         return np.asarray(keras.ops.image.resize(t, (IMG_SIZE, IMG_SIZE)))[0, :, :, 0]
38
39     R, IF, SF = _rsz(R), _rsz(IF), _rsz(SF)
40
41     C, _, _ = np.histogram2d(iq[:, 0], iq[:, 1], bins=IMG_SIZE,
42                             range=[[-2, 2], [-2, 2]], density=True)
43     C = (C / (C.max() + 1e-12)).astype("float32")
44
45     phi = np.angle(x)

```

Figure 3: Inference script, source page 2

```

1     PH, _, _ = np.histogram2d(np.linspace(0, 1, len(phi)), phi,
2                               bins=IMG_SIZE, range=[[0, 1], [-np.pi, np.pi]],
3                               density=True)
4     PH = (PH / (PH.max() + 1e-12)).astype("float32")
5
6     return np.stack([R, IF, SF, C, PH], axis=-1).astype("float32")
7
8 def build_images(iq_batch, chunk_size=512):
9     N = len(iq_batch)
10    out = np.empty((N, IMG_SIZE, IMG_SIZE, 5), dtype="float32")
11    for i in range(0, N, chunk_size):
12        out[i:i + chunk_size] = np.stack([to_spectrogram(f) for f in iq_batch[i:i + chunk_size]])
13    return out
14
15 # — Spectrogram visualisation — matches the notebook's visual sanity check —
16
17 CHANNEL_LABELS = ["Log-Magnitude (R)", "Inst. Frequency (G)", "Spectral Flux (B)",
18                  "Constellation (C)", "Phase-Time (P)"]
19
20 def save_spectrogram_samples(frames, label, out_dir, n_samples=3, seed=42):
21     """Save n_samples randomly chosen frames as 1x5-channel plots to out_dir."""
22     os.makedirs(out_dir, exist_ok=True)
23     rng = np.random.default_rng(seed)
24     indices = rng.choice(len(frames), size=min(n_samples, len(frames)), replace=False)
25
26     for rank, frame_idx in enumerate(indices):
27         fig, axes = plt.subplots(1, 5, figsize=(15, 3))
28         s = to_spectrogram(frames[frame_idx])[np.newaxis] # (1, H, W, 5)
29
30         for ch, ax in enumerate(axes):
31             img = np.asarray(keras.ops.image.resize(s, (IMG_SIZE, IMG_SIZE)))[0, :, :, ch]
32             ax.imshow(img, aspect='auto', origin='lower', cmap='viridis')
33             ax.set_title(CHANNEL_LABELS[ch], fontsize=10)
34             ax.axis('off')
35
36         # show class label on the left edge of the first channel, matching the notebook style
37         axes[0].set_ylabel(label, fontsize=9, rotation=0, labelpad=55, va='center')
38         axes[0].axis('on')
39         axes[0].set_xticks([])
40         axes[0].set_yticks([])
41
42         plt.tight_layout()
43         path = os.path.join(out_dir, f"ota_frame_{frame_idx:05d}_sample{rank + 1}.png")
44         plt.savefig(path, dpi=100, bbox_inches='tight')
45         plt.close(fig)

```

Figure 4: Inference script, source page 3

```

1         print(f"Saved spectrogram : {path}")
2
3 # — Entry point —————
4
5 def main():
6     if len(sys.argv) != 3:
7         sys.exit("Usage: python infer_ota.py <normalized.iq> <baseline_cnn.weights.h5>")
8
9     iq_path      = sys.argv[1]
10    weights_path = sys.argv[2]
11
12    if not os.path.exists(iq_path):
13        sys.exit(f"IQ file not found: {iq_path}")
14    if not os.path.exists(weights_path):
15        sys.exit(f"Weights file not found: {weights_path}")
16
17    frames = np.fromfile(iq_path, dtype=np.float32).reshape(-1, FRAME_LEN, 2)
18    print(f"Loaded : {frames.shape[0]:,} frames from {iq_path}")
19
20    model = build_baseline_cnn((IMG_SIZE, IMG_SIZE, 5), len(CLASSES))
21    model.load_weights(weights_path)
22    print(f"Weights : loaded from {weights_path}")
23
24    print(f"Building {frames.shape[0]:,} spectrograms ...")
25    imgs = build_images(frames)
26    probs = model.predict(imgs, batch_size=BATCH_SIZE, verbose=0)
27    preds = probs.argmax(axis=1)
28
29    print(f"\n{'Frame':>7}  {'Class':<12}  {'Confidence':>10}")
30    print("-" * 34)
31    for i, (pred, prob) in enumerate(zip(preds, probs)):
32        print(f"{i:>7}  {CLASSES[pred]:<12}  {prob[pred]:>9.1%}")
33
34    vote_idx, vote_count = Counter(preds).most_common(1)[0]
35    mean_conf = probs[preds == vote_idx, vote_idx].mean()
36    print(f"\nClassification : {CLASSES[vote_idx]}")
37    print(f"Majority vote : {vote_count}/{len(preds)} frames (mean confidence {mean_conf:.1%})")
38
39    # Save 3 randomly chosen input spectrograms using the same layout as the
40    # notebook's visual sanity check, labelled with the majority-vote class.
41    out_dir = os.path.join(os.path.dirname(os.path.abspath(iq_path)), "ota_rf_input")
42    save_spectrogram_samples(frames, CLASSES[vote_idx], out_dir, n_samples=3)
43
44 if __name__ == "__main__":
45     main()

```

Figure 5: Inference script, source page 4

```

1 # Normalise a GNU Radio complex64 .iq capture to RadioML 2018.01A unit-variance
2 # convention and save as a flat complex64 binary ready for to_spectrogram().
3
4 import os
5 import sys
6 import numpy as np
7
8 INPUT_FILE = os.path.expanduser("fm_c96.3M_s400k.iq")
9 OUTPUT_FILE = os.path.expanduser("fm_c96.3M_s400k_normalized.iq")
10 FRAME_LEN = 1024 # RadioML 2018.01A frame length
11
12 # — Load —————
13
14 if not os.path.exists(INPUT_FILE):
15     sys.exit(f"File not found: {INPUT_FILE}")
16
17 # GNU Radio File Sink writes interleaved Re/Im as complex64 (fc32)
18 raw = np.fromfile(INPUT_FILE, dtype=np.complex64)
19
20 n_frames = len(raw) // FRAME_LEN
21 if n_frames == 0:
22     sys.exit(f"File too short: need at least {FRAME_LEN} samples, got {len(raw):,}")
23
24 # Trim to exact multiple then split into (N, 1024, 2) float32 matching f['X'] in RadioML HDF5
25 frames_c = raw[: n_frames * FRAME_LEN].reshape(n_frames, FRAME_LEN)
26 frames_iq = np.stack([frames_c.real, frames_c.imag], axis=-1) # (N, 1024, 2) float32
27
28 print(f"Loaded : {INPUT_FILE}")
29 print(f"Samples : {len(raw):,} → {n_frames:,} frames of {FRAME_LEN}")
30
31 # — Per-frame unit-variance normalisation —————
32 # O'Shea et al. (2018) state RadioML examples are "normalized to unit variance".
33 # For a zero-mean complex signal,  $\text{std}(I+jQ) = \sqrt{\text{var}(I) + \text{var}(Q)}$ .
34 # Dividing each frame by its std makes total complex power = 1 per frame,
35 # placing IQ amplitude in the [-2, 2] range expected by to_spectrogram()'s
36 # constellation channel histogram.
37
38 z = frames_iq[:, :, 0] + 1j * frames_iq[:, :, 1] # (N, 1024) complex
39 std = np.std(z, axis=1).reshape(n_frames, 1, 1).astype(np.float32) # (N, 1, 1)
40 frames_norm = frames_iq / (std + 1e-12) # broadcast over (1024, 2)
41
42 # — Sanity check —————
43
44 z_norm = frames_norm[:, :, 0] + 1j * frames_norm[:, :, 1]
45 per_frame_std = np.std(z_norm, axis=1)

```

Figure 6: Normalisation script, source page 1

```

1 print(f"Per-frame std - mean: {per_frame_std.mean():.6f} std: {per_frame_std.std():.6f} (target mean ≈ 1.0)")
2
3 low_power = (per_frame_std < 0.01).sum()
4 if low_power:
5     # Frames with near-zero power are dead air or HackRF dropout - warn but keep
6     print(f"Warning: {low_power} frames have std < 0.01 (dead air / dropout). Consider filtering these out.")
7
8 # — Save —————
9 # Write flat complex64 binary - identical format to the input file.
10 # Load in notebook with:
11 # frames = np.fromfile(OUTPUT_FILE, dtype=np.float32).reshape(-1, 1024, 2)
12
13 out = (frames_norm[:, :, 0] + 1j * frames_norm[:, :, 1]).astype(np.complex64)
14 out.ravel().tofile(OUTPUT_FILE)
15
16 print(f"Saved : {OUTPUT_FILE}")
17 print(f"Shape : {frames_norm.shape} (frames, samples, I/Q)")
18 print()
19 print("Load in notebook with:")
20 print(" frames = np.fromfile(OUTPUT_FILE, dtype=np.float32).reshape(-1, 1024, 2)")
21 print(" imgs = build_images(frames)")
22 print(" preds = model_predict(imgs, batch_size=BATCH_SIZE).argmax(axis=1)")

```

Figure 7: Normalisation script, source page 2

```

1 # --- Notebook code cell 2 ---
2 # Install required dependencies for the project
3 %pip install -U numpy h5py matplotlib scipy scikit-learn keras jax jaxlib kagglehub python-dotenv
4 # --- Notebook code cell 3 ---
5 import os
6 # Backend Configuration
7 # I use the JAX backend for Keras as discussed in Chollet and Watson, 2026.
8 os.environ.setdefault("KERAS_BACKEND", "jax")
9
10 from dotenv import load_dotenv
11 import glob
12 import numpy as np
13 import h5py
14 import matplotlib.pyplot as plt
15 from scipy.signal import stft
16 from sklearn.model_selection import train_test_split
17 from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
18 import keras
19 from keras import layers
20 import kagglehub
21 import math
22
23 # Verify the setup
24 print(f"Keras version: {keras.__version__} | Backend: {keras.backend.backend()}")
25
26 # -----
27 # CONFIGURATION
28 # -----
29
30 # Load Kaggle API keys
31 load_dotenv()
32
33 # Authenticate with Kaggle to access the dataset
34 token = os.getenv("KAGGLE_API_TOKEN")
35 token_path = os.path.expanduser(os.getenv("KAGGLE_API_TOKEN_PATH", "~/kaggle/access_token"))
36
37 # Set these for the Kaggle library to find automatically
38 os.environ["KAGGLE_API_TOKEN"] = token
39 os.environ["KAGGLE_API_TOKEN_PATH"] = token_path
40
41 # Dataset Source: DeepSig RadioML 2018.01A
42 KAGGLE_DATASET = "pinxau1000/radioml2018"
43 HDF5_FILENAME = "GOLD_XYZ_OSC.0001_1024.hdf5"
44
45 # Allow overriding the path if already have the file locally (saves re-downloading)

```

Figure 8: Notebook source, page 1

```

1 RADIOML_PATH      = os.environ.get("RADIOML_PATH")
2
3 # -----
4 # TRAINING HYPERPARAMETERS
5 # -----
6
7 # Data Structure
8 FRAMES_PER_BLOCK = 106496 # Total frames per block (26 SNRs * 4096 frames/mod)
9 FRAMES_PER_CELL  = 4096   # Frames per (Modulation, SNR) pair
10 N_SNR            = 26     # Range: -20 dB to +30 dB in 2 dB steps
11
12 # Prototype subsample per cell
13 SAMPLES_PER_CELL = 200
14
15 # Image Resolution
16 IMG_SIZE         = 128
17
18 # Batch size for training and evaluation
19 # Process 32 samples at a time. Default for image processing (Chollet and Watson, 2026) is 64,
20 # but I reduced it to 32 to avoid OOM errors on my hardware.
21 BATCH_SIZE       = 32
22
23 # Setting seeds ensures that random splits and weight initializations are identical every time
24 SEED             = 42
25 np.random.seed(SEED)
26 keras.utils.set_random_seed(SEED)
27
28 # -----
29 # EXECUTION: Check Local Path
30 # -----
31 if RADIOML_PATH and not os.path.exists(RADIOML_PATH):
32     print(f"Warning: Local path specified but file not found: {RADIOML_PATH}")
33     RADIOML_PATH = None
34
35 print(f"\nConfiguration Loaded:")
36 print(f"- Dataset: RadioML 2018.01A ({KAGGLE_DATASET})")
37 print(f"- Target Size: {IMG_SIZE}x{IMG_SIZE} spectrograms")
38 print(f"- Subsample Count: ~{SAMPLES_PER_CELL * 8 * N_SNR} frames")
39 print(f"- Random Seed: {SEED}")
40 # --- Notebook code cell 5 ---
41 # -----
42 # DATA SET SETUP
43 # -----
44
45 def resolve_dataset_path():

```

Figure 9: Notebook source, page 2

```

1  # If 'RADIOML_PATH' is already set in the environment, I use that location first.
2  if RADIOML_PATH and os.path.exists(RADIOML_PATH):
3      print(f"[INFO] Using local dataset override: {RADIOML_PATH}")
4      return RADIOML_PATH
5
6  # If no local path is set, download from Kaggle.
7  # kagglehub caches the dataset under ~/.cache/kagglehub automatically.
8  # This ensures the heavy 21.5 GB pull happens only once per machine.
9  try:
10     folder = kagglehub.dataset_download(KAGGLE_DATASET)
11
12     # Search recursively for the specific HDF5 file within the downloaded folder
13     hits = glob.glob(os.path.join(folder, "**", HDF5_FILENAME), recursive=True)
14
15     if not hits:
16         raise FileNotFoundError(
17             f"Required file '{HDF5_FILENAME}' not found in downloaded content."
18         )
19
20     path = hits[0]
21     print(f"[INFO] Using Kaggle dataset (cached): {path}")
22     return path
23
24 except Exception as e:
25     # Provide guidance if authentication fails
26     raise RuntimeError(
27         "Failed to download dataset. Please check:\n"
28         "1. Kaggle API Token is valid (~/.kaggle/kaggle.json)\n"
29         "2. Environment variables KAGGLE_USERNAME/KAGGLE_KEY are set\n"
30         f"Original Error: {str(e)}"
31     )
32
33 # -----
34 # EXECUTION: Resolve Dataset Path
35 # -----
36 DATASET_PATH = resolve_dataset_path()
37 print(f"\n[SUCCESS] Dataset ready at: {DATASET_PATH}")
38 # --- Notebook code cell 7 ---
39 # -----
40 # MODULATION CLASS SELECTION
41 # -----
42
43 # Base classes from RadioML 2018.01A paper [O'Shea et al., 2018].
44 # This list contains all 24 modulation schemes available in the dataset.
45 base_modulation_classes = [

```

Figure 10: Notebook source, page 3

```

1     'OOK', '4ASK', '8ASK', 'BPSK', 'QPSK', '8PSK', '16PSK', '32PSK',
2     '16APSK', '32APSK', '64APSK', '128APSK', '16QAM', '32QAM', '64QAM',
3     '128QAM', '256QAM', 'AM-SSB-WC', 'AM-SSB-SC', 'AM-DSB-WC', 'AM-DSB-SC',
4     'FM', 'GMSK', 'OQPSK'
5 ]
6
7 # I reduced the set from 24 to 8 classes:
8 selected_modulation_classes = ['4ASK', 'BPSK', 'QPSK', '16PSK', '16QAM', 'FM', 'AM-DSB-WC', '32APSK']
9
10 # Map selected names to their original integer IDs in the dataset
11 selected_ids = [base_modulation_classes.index(c) for c in selected_modulation_classes]
12
13 print(f"Selected {len(selected_ids)} classes out of {len(base_modulation_classes)}:")
14 print(f"IDs: {selected_ids}")
15 print(f"Classes: {selected_modulation_classes}")
16
17 # -----
18 # DATA LOADING FUNCTION
19 # -----
20
21 def load_cells(path, mod_ids, samples_per_cell):
22     X, y, snr = [], [], []
23
24     print(f"\nLoading data for {len(mod_ids)} classes...")
25
26     with h5py.File(path, "r") as f:
27         # Loop through each chosen modulation class
28         for new_label, mid in enumerate(mod_ids):
29
30             # Loop through each SNR level (-20 dB to +30 dB)
31             for s in range(N_SNR):
32
33                 # Calculate start index for specific (modulation, SNR) cell
34                 start = mid * FRAMES_PER_BLOCK + s * FRAMES_PER_CELL
35
36                 # Randomly select 'samples_per_cell' frames without replacement
37                 pick = np.sort(np.random.choice(FRAMES_PER_CELL, samples_per_cell, replace=False))
38
39                 # Compute global indices
40                 idx = (start + pick).tolist()
41
42                 # Load Data
43                 # f['X']: I/Q samples (shape: N, 1024, 2)
44                 # f['Z']: SNR values
45                 X.append(f['X'][idx])

```

Figure 11: Notebook source, page 4

```

1         snr.append(f['Z'][idx].ravel())
2
3         # Create Labels (0 to len(mod_ids)-1)
4         y.append(np.full(samples_per_cell, new_label))
5
6     # Combine all lists into single NumPy arrays
7     # X: (Total_Samples, 1024, 2)
8     # y: (Total_Samples,)
9     # snr: (Total_Samples,)
10    return np.concatenate(X), np.concatenate(y), np.concatenate(snr)
11
12    # -----
13    # EXECUTION: Load Prototype Dataset
14    # -----
15
16    # Load the subsampled data using the defined parameters
17    X_iq, y_int, snr = load_cells(DATASET_PATH, selected_ids, SAMPLES_PER_CELL)
18
19    # Verify dimensions
20    print(f"\nData Loaded:")
21    print(f"- I/Q Tensor Shape: {X_iq.shape} (Samples, Time, I/Q)")
22    print(f"- Labels Shape: {y_int.shape}")
23    print(f"- Unique SNR Levels: {np.unique(snr)}")
24    print(f"- Total Samples: {X_iq.shape[0]}")
25    # --- Notebook code cell 9 ---
26    # -----
27    # SNR FILTER: retain only SNR >= -2 dB (follows Waqas et al. 2023)
28    # -----
29    SNR_THRESHOLD = -2
30    filtered_indices = np.where(snr >= SNR_THRESHOLD)[0]
31    y_int_filt = y_int[filtered_indices]
32    snr_filt = snr[filtered_indices]
33
34    print(f"SNR filter (>= {SNR_THRESHOLD} dB): {len(filtered_indices):,} / {len(snr):,} samples retained")
35    print(f"Retained SNR Levels: {np.unique(snr_filt)}")
36
37    # -----
38    # DATA SPLITTING (on filtered global indices)
39    # -----
40
41    # stratify key preserves (modulation, SNR) balance across all three sets
42    stratify_key = [f"{a}_{b}" for a, b in zip(y_int_filt, snr_filt)]
43
44    # Split positions within filtered_indices, then map back to global positions
45    # so idx_train / idx_val / idx_test can directly index img_paths, y_int, snr.

```

Figure 12: Notebook source, page 5

```

1 pos_all = np.arange(len(filtered_indices))
2
3 pos_train, pos_tmp = train_test_split(
4     pos_all,
5     test_size=0.30,
6     random_state=SEED,
7     stratify=stratify_key
8 )
9
10 stratify_tmp = [stratify_key[i] for i in pos_tmp]
11 pos_val, pos_test = train_test_split(
12     pos_tmp,
13     test_size=0.50,
14     random_state=SEED,
15     stratify=stratify_tmp
16 )
17
18 idx_train = filtered_indices[pos_train]
19 idx_val = filtered_indices[pos_val]
20 idx_test = filtered_indices[pos_test]
21
22 # -----
23 # EXECUTION: Confirm Dataset Sizes and SNR Coverage
24 # -----
25 print("\nFinal Dataset Sizes")
26 total = len(filtered_indices)
27 print(f"Training Set: {len(idx_train):,} samples (~{len(idx_train)/total*100:.1f}%)")
28 print(f"Validation Set: {len(idx_val):,} samples (~{len(idx_val)/total*100:.1f}%)")
29 print(f"Test Set: {len(idx_test):,} samples (~{len(idx_test)/total*100:.1f}%)")
30
31 def check_snr_coverage(indices, name):
32     subset_snr = snr[indices]
33     print(f"{name} SNR Range: [{subset_snr.min():.0f}, {subset_snr.max():.0f}] dB")
34
35 check_snr_coverage(idx_train, "Train")
36 check_snr_coverage(idx_val, "Val ")
37 check_snr_coverage(idx_test, "Test ")
38 # --- Notebook code cell 12 ---
39 class SpectrogramSequence(keras.utils.Sequence):
40     def __init__(self, iq_data, labels, batch_size=BATCH_SIZE, **kwargs):
41         super().__init__(**kwargs)
42         self.iq_data = iq_data
43         self.labels = labels
44         self.batch_size = batch_size

```

Figure 13: Notebook source, page 6

```

1  def __len__(self):
2      return math.ceil(len(self.iq_data) / self.batch_size)
3
4  def __getitem__(self, idx):
5      lo = idx * self.batch_size
6      hi = lo + self.batch_size
7      X = np.stack([to_spectrogram(f) for f in self.iq_data[lo:hi]])
8      return X, self.labels[lo:hi]
9  # --- Notebook code cell 14 ---
10 # -----
11 # SPECTROGRAM GENERATION PIPELINE
12 # -----
13
14 def to_spectrogram(iq, nperseg=128, noverlap=112):
15     # I/Q data is combined into one complex signal.
16     x = iq[:, 0] + 1j * iq[:, 1]
17
18     # Short-Time Fourier Transform (STFT):
19     _, _, Z = stft(x, window='hann', nperseg=nperseg, noverlap=noverlap,
20                   return_onesided=False, boundary=None, padded=False)
21
22     # Center Zero Frequency:
23     Z_shifted = np.fft.fftshift(Z, axes=0)
24     S = np.abs(Z_shifted)
25
26     # Channel R: log-magnitude spectrogram
27     R_raw = 20.0 * np.log10(S + 1e-12)
28     R = (R_raw - R_raw.min()) / (R_raw.max() - R_raw.min() + 1e-12)
29
30     # Channel G: instantaneous frequency - dφ/dt along time axis.
31     # Edge-pad (repeat first diff) so the first frame carries real signal
32     # rather than a structural zero the model learns to ignore.
33     phi_t = np.unwrap(np.angle(Z_shifted), axis=1)
34     d_phi_t = np.diff(phi_t, axis=1)
35     IF = np.concatenate([d_phi_t[:, :1], d_phi_t], axis=1)
36     if_lo, if_hi = np.percentile(IF, 1), np.percentile(IF, 99)
37     IF = np.clip(IF, if_lo, if_hi)
38     IF = (IF - if_lo) / (if_hi - if_lo + 1e-12)
39
40     # Channel B: spectral flux - d(log-mag)/dt along time axis (amplitude velocity).
41     # Captures energy-transition dynamics: FM sweep band, PSK/QAM symbol transitions.
42     # Fisher ratio at +30 dB is 23% higher than group delay; visually structured vs GD noise.
43     d_R = np.diff(R_raw, axis=1)
44     SF = np.concatenate([d_R[:, :1], d_R], axis=1)
45     sf_lo, sf_hi = np.percentile(SF, 1), np.percentile(SF, 99)

```

Figure 14: Notebook source, page 7

```

1     SF = np.clip(SF, sf_lo, sf_hi)
2     SF = (SF - sf_lo) / (sf_hi - sf_lo + 1e-12)
3
4     # resize STFT channels to square before stacking with non-STFT channels
5     def _rsz(arr):
6         t = arr[np.newaxis, :, :, np.newaxis]
7         return np.asarray(keras.ops.image.resize(t, (IMG_SIZE, IMG_SIZE)))[0, :, :, 0]
8
9     R, IF, SF = _rsz(R), _rsz(IF), _rsz(SF)
10
11    # Channel 4: 2D IQ histogram – encodes constellation geometry directly.
12    C, _, _ = np.histogram2d(iq[:, 0], iq[:, 1], bins=IMG_SIZE,
13                             range=[[-2, 2], [-2, 2]], density=True)
14    C = (C / (C.max() + 1e-12)).astype("float32")
15
16    # Channel 5: phase clusters at N discrete angles for N-PSK/QAM.
17    phi = np.angle(x)
18    PH, _, _ = np.histogram2d(np.linspace(0, 1, len(phi)), phi,
19                              bins=IMG_SIZE, range=[[0, 1], [-np.pi, np.pi]],
20                              density=True)
21    PH = (PH / (PH.max() + 1e-12)).astype("float32")
22
23    return np.stack([R, IF, SF, C, PH], axis=-1).astype("float32")
24
25    def build_images(iq_batch, chunk_size=512):
26        N = len(iq_batch)
27        out = np.empty((N, IMG_SIZE, IMG_SIZE, 5), dtype="float32")
28        for i in range(0, N, chunk_size):
29            out[i:i + chunk_size] = np.stack([to_spectrogram(f) for f in iq_batch[i:i + chunk_size]])
30        return out
31
32    # -----
33    # EXECUTION: Generate Spectrograms and One-Hot Labels
34    # -----
35
36    # Convert labels to one-hot encoding
37    y_oh = keras.utils.to_categorical(y_int, num_classes=len(selected_ids))
38
39    # Convert raw I/Q data to spectrograms generated on-the-fly during training using a custom Keras Sequence.
40    train_seq = SpectrogramSequence(X_iq[idx_train], y_oh[idx_train], batch_size=BATCH_SIZE)
41    val_seq = SpectrogramSequence(X_iq[idx_val], y_oh[idx_val], batch_size=BATCH_SIZE)
42    # --- Notebook code cell 16 ---
43    # -----
44    # SPECTROGRAM VISUALIZATION – All 5 Channels per Modulation Class
45    # -----

```

Figure 15: Notebook source, page 8

```

1 CHANNEL_LABELS = ["Log-Magnitude (R)", "Inst. Frequency (G)", "Spectral Flux (B)",
2                  "Constellation (C)", "Phase-Time (P)"]
3 n_classes = len(selected_ids)
4
5 fig, axes = plt.subplots(n_classes, 5, figsize=(15, 3 * n_classes))
6
7 for cls in range(n_classes):
8     high_snr_indices = np.where(y_int == cls & (snr >= 28))[0]
9     j = high_snr_indices[0] if len(high_snr_indices) > 0 else np.where(y_int == cls)[0][0]
10
11     for ch in range(5):
12         ax = axes[cls, ch]
13         s = to_spectrogram(X_iq[j])[np.newaxis] # (1, H, W, 5)
14         ax.imshow(np.asarray(keras.ops.image.resize(s, (IMG_SIZE, IMG_SIZE)))[0, :, :, ch], aspect='auto', origin='lower', cmap='viridis')
15         ax.axis('off')
16         if cls == 0:
17             ax.set_title(CHANNEL_LABELS[ch], fontsize=10)
18         if ch == 0:
19             # ylabel places the class name to the left of the row
20             ax.set_ylabel(selected_modulation_classes[cls], fontsize=9,
21                           rotation=0, labelpad=55, va='center')
22         ax.axis('on')
23         ax.set_xticks([])
24         ax.set_yticks([])
25
26 plt.tight_layout()
27 plt.show()
28 # --- Notebook code cell 19 ---
29 metrics=[keras.metrics.F1Score(average='macro', name='macro_f1')]
30 # --- Notebook code cell 20 ---
31 # -----
32 # BASELINE CNN ARCHITECTURE
33 # -----
34
35 # padding='same' preserves spatial dimensions so no pooling adjustment is needed.
36 def residual_block(x, filters):
37     shortcut = layers.Conv2D(filters, 1, padding='same')(x)
38     x = layers.Conv2D(filters, 3, padding='same', activation='relu')(x)
39     x = layers.Conv2D(filters, 3, padding='same')(x)
40     return layers.Activation('relu')(layers.Add()([x, shortcut]))
41
42 def build_baseline_cnn(input_shape, n_classes):
43
44     # Define the Input Layer

```

Figure 16: Notebook source, page 9

```

1  inputs = keras.Input(shape=input_shape)
2
3  # Convolutional layer. 16 filters, each scanning 3x3 pixel areas.
4  # 'relu' lets the network make yes/no decisions so it can understand
5  # complicated features like edges and shapes, not just simple ones.
6  x = layers.Conv2D(16, 3, activation='relu')(inputs)
7
8  # MaxPooling downsamples images speeding up processing
9  # and helping the network focus on important features
10 # regardless of where they appear in the picture.
11 x = layers.MaxPooling2D()(x)
12
13 # Middle layer feature extraction
14 # Increasing filters to 32 so the network can find richer, more detailed patterns
15 # as it gets deeper and understands more about what it's seeing.
16 x = layers.Conv2D(32, 3, activation='relu')(x)
17 x = layers.MaxPooling2D()(x)
18
19 # Deep layer feature extraction
20 # Bumping up to 64 filters to detect the most complex, high-level features
21 # to recognize full object parts rather than simple shapes.
22 x = layers.Conv2D(64, 3, activation='relu')(x)
23 x = layers.MaxPooling2D()(x)
24
25 # Residual block: helps the network learn better by allowing it to skip layers if needed.
26 x = residual_block(x, 64)
27
28 # 4th block reduces 14x14 spatial map (from 128px input) to 6x6 before flattening.
29 x = layers.Conv2D(128, 3, activation='relu')(x)
30 x = layers.MaxPooling2D()(x)
31
32 # Dense head classification
33 # Flatten reshapes all those 2D feature maps into a single long list
34 # so the final prediction layers can read them easily.
35 x = layers.Flatten()(x)
36
37 # Dropout randomly turns off 10% of the neurons during training,
38 # which helps the model generalize better and not memorize the training data.
39 x = layers.Dropout(0.1)(x)
40
41 # Single dense layer with 256 neurons squeezes all the features together,
42 # mixing and combining them so the model can make its final prediction.
43 x = layers.Dense(256, activation='relu')(x)
44
45 # Output layer. One neuron for each modulation class, using 'softmax' activation.

```

Figure 17: Notebook source, page 10

```

1  # Softmax turns raw scores into probabilities that all add up to 100%.
2  outputs = layers.Dense(n_classes, activation='softmax')(x)
3
4  # Create the model instance linking inputs to outputs.
5  model = keras.Model(inputs, outputs, name="baseline_cnn")
6
7  # Set up how the model will train.
8  # Adam optimizer: Automatically adjusts learning speed, starts at a recommended default (Chollet and Watson, 2026).
9  # Categorical crossentropy: Measures how wrong the guesses are across multiple categories.
10 # Accuracy: Tracks what % of predictions the model gets right.
11 model.compile(
12     optimizer=keras.optimizers.Adam(learning_rate=1e-3),
13     loss=keras.losses.CategoricalFocalCrossentropy(gamma=2.0),
14     # Macro average weights each class equally, penalising confusion between similar classes.
15     metrics=metrics
16 )
17
18 return model
19
20 # Instantiate the model dynamically based on dataset's
21 # image shape and 8 unique modulation classes selected.
22 model = build_baseline_cnn((IMG_SIZE, IMG_SIZE, 5), len(selected_ids))
23
24 # -----
25 # EXECUTION: Evaluate model architecture summary
26 # -----
27
28 model.summary()
29 # --- Notebook code cell 22 ---
30 # -----
31 # TRAINING
32 # -----
33
34 callbacks = [
35     # Early Stopping
36     # Watches validation loss during training. If it doesn't
37     # get better for 8 rounds in a row, training ends right away.
38     keras.callbacks.EarlyStopping(
39         monitor='val_macro_f1',      # Watch the validation macro F1 score metric
40         patience=5,                  # Wait 5 epochs before stopping
41         mode='max',                  # Monitor for maximum since I want to maximize macro F1 score
42         restore_best_weights=True    # Roll back to the best epoch's weights
43     ),
44
45     # Best-Only Checkpointing

```

Figure 18: Notebook source, page 11

```

1  # Saves the model's weights only when 'val_macro_f1' improves.
2  # This ensures the weights file 'baseline_cnn.weights.h5'
3  # always contains the single best performing version seen during training.
4  keras.callbacks.ModelCheckpoint(
5      filepath='baseline_cnn.weights.h5',
6      monitor='val_macro_f1',
7      save_best_only=True,      # Only overwrite if performance gets better
8      save_weights_only=True   # Save only weights to save disk space
9  ),
10
11  # Learning Rate Reduction on Plateau
12  # If validation macro F1 score stops improving for 4 epochs, cut the learning rate in half.
13  # Smaller steps let the model carefully fine-tune when it's stuck and can't improve.
14  # 'min_lr=2.5e-5' stops the rate from dropping so low it practically freezes.
15  keras.callbacks.ReduceLROnPlateau(
16      monitor='val_macro_f1',
17      factor=0.5,               # Reduce LR by 50%
18      patience=4,              # Trigger reduction after 4 epochs of no improvement
19      mode='max',              # Monitor for maximum since I want to maximize macro F1 score
20      min_lr=1e-6              # Minimum learning rate allowed
21  ),
22 ]
23
24 # -----
25 # EXECUTION: Model Training
26 # -----
27
28 # -----
29 # NOTE: The test set (idx_test) is completely untouched here. It is reserved
30 # solely for the final unbiased evaluation after all hyperparameter tuning is done.
31 # -----
32
33 training_inputs = SpectrogramSequence(X_iq[idx_train], y_oh[idx_train])
34 validation_inputs = SpectrogramSequence(X_iq[idx_val], y_oh[idx_val])
35
36 history = model.fit(
37     training_inputs,           # Training inputs
38     validation_data=validation_inputs, # Validation split (monitored by callbacks)
39     epochs=40,                # Maximum possible epochs (EarlyStopping may stop earlier)
40     callbacks=callbacks,      # Attach safety callbacks
41     verbose=2                 # Print training progress per epoch
42 )
43 # --- Notebook code cell 24 ---
44 # -----
45 # TRAINING HISTORY VISUALIZATION

```

Figure 19: Notebook source, page 12

Figure B19. *final.ipynb* executable source, page 12.

```
1 # -----
2
3 h = history.history
4
5 fig, (a1, a2) = plt.subplots(1, 2, figsize=(11, 4))
6
7 # Macro F1 Curve
8 a1.plot(h['macro_f1'], label='train')
9 a1.plot(h['val_macro_f1'], label='val')
10
11 # Random-guess F1 baseline for n equal classes.
12 a1.axhline(1 / len(selected_ids), ls='--', c='grey', label='random 12.5%')
13
14 a1.set_title('Model Macro F1')
15 a1.set_xlabel('Epoch')
16 a1.legend(loc='lower right')
17
18 # Loss Curve
19 a2.plot(h['loss'], label='train')
20 a2.plot(h['val_loss'], label='val')
21
22 a2.set_title('Model Loss')
23 a2.set_xlabel('Epoch')
24 a2.legend(loc='upper right')
25
26 # -----
27 # EXECUTION: Render training history plots
28 # -----
29
30 plt.tight_layout()
31 plt.show()
32 # --- Notebook code cell 27 ---
33 # -----
34 # EVALUATION ON TEST SET
35 # -----
36 test_inputs = SpectrogramSequence(X_iq[idx_test], y_oh[idx_test])
37
38 test_loss, test_acc = model.evaluate(
39     test_inputs,
40     verbose=0 # Silent mode to keep output clean
41 )
42
43 # Prepare ground truth labels
44 y_true = y_int[idx_test]
```

Figure 20: Notebook source, page 13

Figure B20. *final.ipynb* executable source, page 13.

Figure B21. *final.ipynb* executable source, page 14.

Figure B22. *final.ipynb* executable source, page 15.

Figure B23. *final.ipynb* executable source, page 16.

Figure B24. *final.ipynb* executable source, page 17.

```

1 # Predict class probabilities and take argmax to get predicted class indices
2 y_pred = model.predict(test_inputs, verbose=0).argmax(1)
3
4 # Compute the raw confusion matrix counts
5 cm = confusion_matrix(y_true, y_pred)
6
7 # Visualize the matrix to spot diagonal dominance (correct predictions)
8 # and off-diagonal hotspots (systematic errors).
9 # 'selected_modulation_classes' ensures the axis labels match our dataset.
10 ConfusionMatrixDisplay(
11     cm,
12     display_labels=selected_modulation_classes
13 ).plot(
14     xticks_rotation=45, # Rotate labels for readability
15     cmap='Blues'        # Blue colormap highlights intensity
16 )
17
18 plt.tight_layout()
19 plt.show()
20
21 # -----
22 # METRICS PREPARATION
23 # -----
24
25 # Evaluate Model
26 test_loss, test_acc = model.evaluate(test_inputs, verbose=0)
27 floor = 1 / len(selected_ids)
28
29 # Prepare Labels and Predictions
30 y_true = y_int[idx_test]
31 y_pred = model.predict(test_inputs, verbose=0).argmax(axis=1)
32 cm = confusion_matrix(y_true, y_pred)
33 labels_list = list(selected_modulation_classes)
34 n_classes = len(labels_list)
35
36 # Calculate Per-Class Metrics (Precision, Recall, F1)
37 metrics_data = []
38 for i in range(n_classes):
39     support = int(cm[i].sum())
40     tp = int(cm[i, i])
41     fp = int(cm[:, i].sum()) - tp
42     fn = int(cm[i].sum()) - tp
43
44     precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
45     recall    = tp / (tp + fn) if (tp + fn) > 0 else 0.0

```

Figure 21: Notebook source, page 14

```

1     f1_score = 2 * precision * recall / (precision + recall) if (precision + recall) > 0 else 0.0
2
3     metrics_data.append({
4         'class': labels_list[i],
5         'support': support,
6         'precision': precision,
7         'recall': recall,
8         'f1': f1_score
9     })
10
11 # Identify Top 8 Worst Confusions
12 confusions = []
13 for i in range(n_classes):
14     for j in range(n_classes):
15         if i != j and cm[i, j] > 0:
16             confusions.append((labels_list[i], labels_list[j], int(cm[i, j])))
17
18 # Sort by count descending and take top 8
19 confusions.sort(key=lambda x: x[2], reverse=True)
20 worst_pairs = confusions[:8]
21
22 # Per-Class Classification Metrics
23 fig_metrics = plt.figure(figsize=(14, 7))
24 ax_metrics = fig_metrics.add_subplot()
25
26 x = np.arange(len(labels_list))
27 width = 0.25
28
29 # Extract metrics for plotting
30 precisions = [m['precision'] for m in metrics_data]
31 recalls = [m['recall'] for m in metrics_data]
32 f1_scores = [m['f1'] for m in metrics_data]
33
34 # Create grouped bar chart for Precision, Recall, and F1-Score
35 b1 = ax_metrics.bar(x - width, precisions, width, label='Precision', color='#1E88E5', alpha=0.9, edgecolor='white')
36 b2 = ax_metrics.bar(x, recalls, width, label='Recall', color='#43A047', alpha=0.9, edgecolor='white')
37 b3 = ax_metrics.bar(x + width, f1_scores, width, label='F1-Score', color='#FB8C00', alpha=0.9, edgecolor='white')
38
39 # Set axis labels, title, and ticks for clarity
40 ax_metrics.set_ylabel('Score (0.0 - 1.0)', fontsize=12, fontweight='bold')
41 ax_metrics.set_title('Per-Class Classification Performance', fontsize=16, fontweight='bold', pad=15)
42 ax_metrics.set_xticks(x)
43 ax_metrics.set_xticklabels(labels_list, rotation=45, ha='right', fontsize=10)
44 ax_metrics.set_ylim(0, 1.05)
45 ax_metrics.legend(loc='lower right', frameon=True, fancybox=True, shadow=True)

```

Figure 22: Notebook source, page 15

```

1 ax_metrics.grid(axis='y', linestyle='--', alpha=0.3)
2
3 # Add value labels on bars
4 for bars in [b1, b2, b3]:
5     for rect in bars:
6         h = rect.get_height()
7         if h > 0:
8             ax_metrics.text(rect.get_x() + rect.get_width()/2., h, f'{h:.2f}',
9                             ha='center', va='bottom', fontsize=8, fontweight='bold')
10
11 # -----
12 # EXECUTION: Render Per-Class Metrics Bar Chart
13 # -----
14
15 plt.tight_layout()
16 plt.show(fig_metrics)
17
18 # -----
19 # AGENT-READABLE TEXT SUMMARY (no plots)
20 # -----
21
22 lines = []
23 lines.append("=== EVALUATION SUMMARY ===")
24 lines.append(f"Test Accuracy : {test_acc:.4f} | Loss: {test_loss:.4f} | Chance: {floor:.4f}")
25 lines.append(f"Lift over chance: {test_acc / floor:.2f}x\n")
26
27 lines.append(f"{'Class':<12} {'Prec':>6} {'Rec':>6} {'F1':>6} {'N':>6}")
28 lines.append("-" * 42)
29 for m in metrics_data:
30     lines.append(f"{'class':<12} {'precision':>6.3f} {'recall':>6.3f} {'f1':>6.3f} {'support':>6}")
31
32 lines.append(f"\nTop-{{len(worst_pairs)}} confusions (true → pred : count):")
33 for true_cls, pred_cls, count in worst_pairs:
34     lines.append(f"  {true_cls:<10} → {pred_cls:<10} : {count}")
35
36 summary = "\n".join(lines)
37 print(summary)
38 # --- Notebook code cell 30 ---
39 # -----
40 # SNR ANALYSIS
41 # -----
42
43 # Extract the SNR value for each sample in the filtered final-model test set
44 snr_test = snr_filt[pos_test]

```

Figure 23: Notebook source, page 16

```

1 # Identify retained SNR levels present in the final test data
2 # (-2, 0, ..., 30 dB)
3 levels = np.unique(snr_test)
4
5 # Calculate accuracy for each retained SNR level independently
6 acc_by_snr = [
7     (y_pred[snr_test == L] == y_true[snr_test == L]).mean()
8     for L in levels
9 ]
10
11 # Plot the results
12 plt.figure(figsize=(7, 4))
13 plt.plot(
14     levels,
15     acc_by_snr,
16     marker='o',      # Markers make individual points visible
17     linewidth=2,     # Thicker line for readability
18     label='Test Accuracy'
19 )
20
21 # Draw the baseline (random guessing threshold) for reference.
22 # If the model dips below this line, it's performing worse than random.
23 floor = 1 / len(selected_ids)
24 plt.axhline(floor, ls='--', c='grey', alpha=0.7, label=f'Random Guess ({floor:.1%})')
25
26 # Labeling
27 plt.xlabel('Signal-to-Noise Ratio (SNR) [dB]', fontsize=11)
28 plt.ylabel('Test Accuracy', fontsize=11)
29 plt.title('Modulation Classification Accuracy vs. SNR Levels', fontsize=13, fontweight='bold')
30
31 # Add grid to improve readability
32 plt.grid(True, alpha=0.3, linestyle=':')
33
34 # Set y-axis limits for better visibility
35 plt.ylim(bottom=0, top=max(acc_by_snr)*1.05 if acc_by_snr else 1.0)
36
37 # -----
38 # EXECUTION: Render per SNR vs. Random Accuracy Plot
39 # -----
40
41 plt.legend(loc='lower right')
42 plt.tight_layout()
43 plt.show()

```

Figure 24: Notebook source, page 17

Appendix C - Design and workplan diagrams

Figures C1 and C2 support the Design chapter's system-structure and workplan descriptions. Figure C1 is the system architecture referenced in the Project Structure subsection; Figure C2 is the detailed Gantt chart referenced in the same subsection.

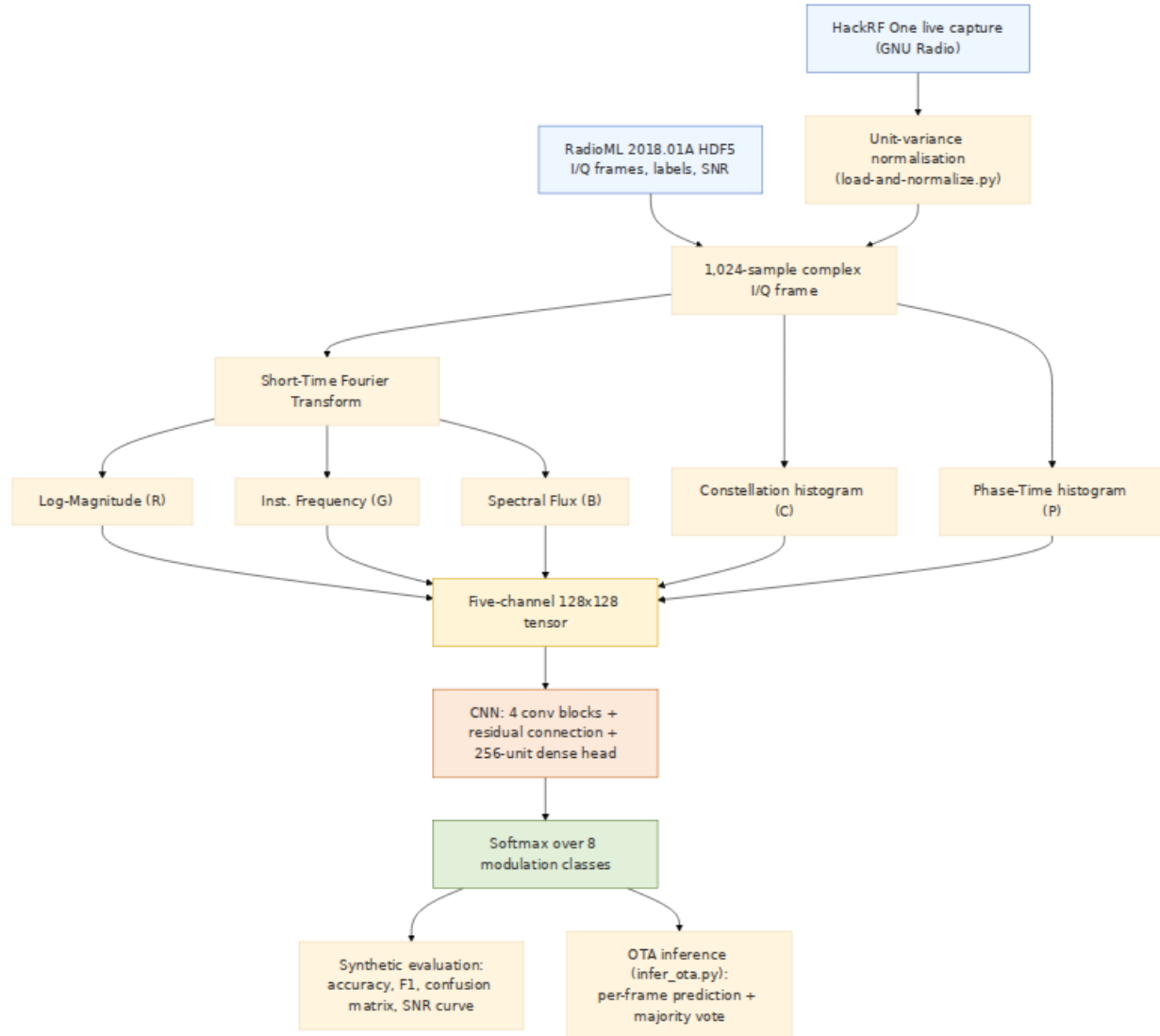


Figure 25: System architecture: preprocessing, five-channel tensor, CNN, and the two evaluation paths

Figure C1. System architecture from raw I/Q input to synthetic evaluation and OTA inference.

Figure C2. Detailed project Gantt chart.

Appendix D - Synthetic evaluation figures

Figures D1-D8 are exported directly from notebook outputs and support the Evaluation chapter's baseline, transfer-learning, and final-model subsections. The baseline figures (D1-D2) document the earlier all-SNR experiment, the transfer-learning figures (D3-D4) document the intermediate pretrained-backbone experiment, and the final-model figures (D5-D8) document

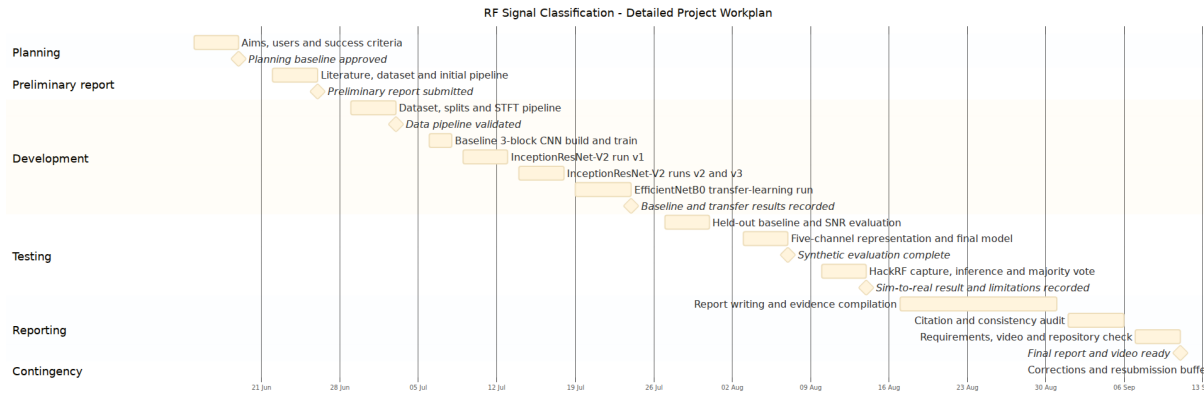


Figure 26: Detailed project Gantt chart

the later five-channel experiment. Together, these outputs make the visual evidence traceable to the successive experiment versions discussed in the report.

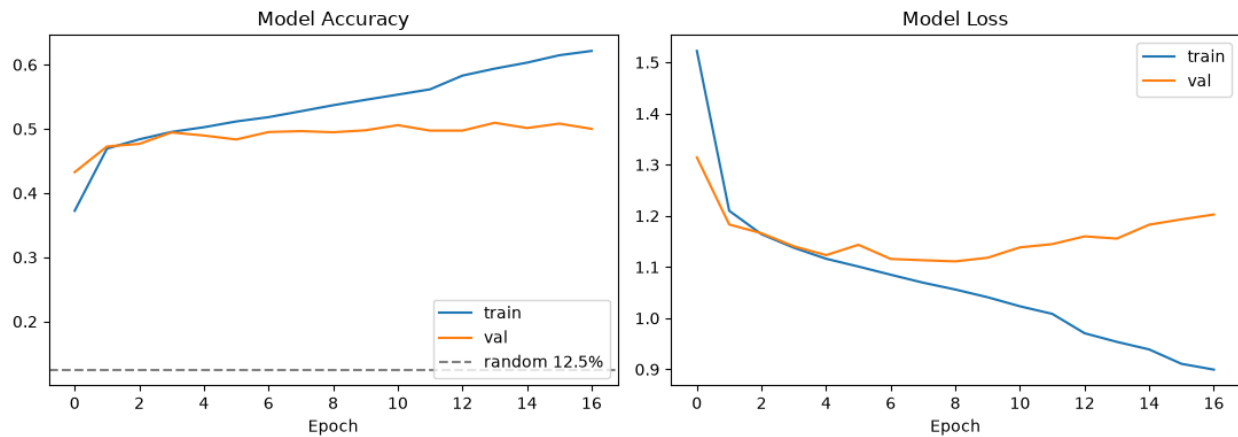


Figure 27: Baseline training history

Figure D1. Baseline CNN training history from the all-SNR experiment. Validation loss reaches its minimum at epoch 9, after which the widening train-validation gap shows overfitting.

Figure D2. Baseline CNN confusion matrix from the all-SNR experiment. The bright 16QAM column is the attractor effect that absorbs the other digital classes.

Figure D3. EfficientNetB0 training history, including the transition from frozen-head training to fine-tuning. Validation accuracy is still rising when the epoch budget ends.

Figure D4. EfficientNetB0 confusion matrix. Errors are spread across the digital classes rather than collapsing onto a single attractor.

Figure D5. Final five-channel CNN training history. Validation macro-F1 plateaus near 0.94 from about epoch 5 while validation loss later rises, which is the pattern the early-stopping and best-weight callbacks exist to catch.

Figure D6. Final five-channel CNN confusion matrix. The attractor column of Figure D2 is gone; the residual errors sit on the 16QAM/32APSK and 16PSK/32APSK pairs.

Figure D7. Final five-channel CNN per-class precision, recall, and F1 scores. The analogue and

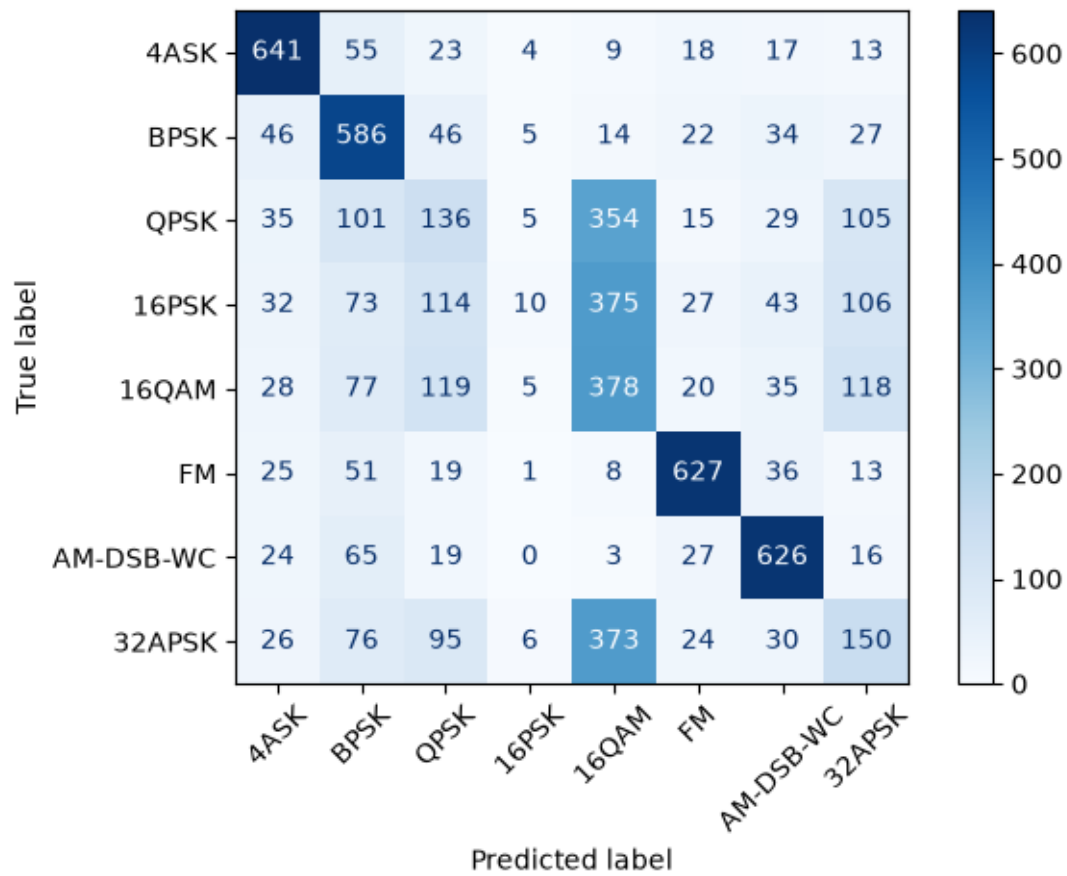


Figure 28: Baseline confusion matrix

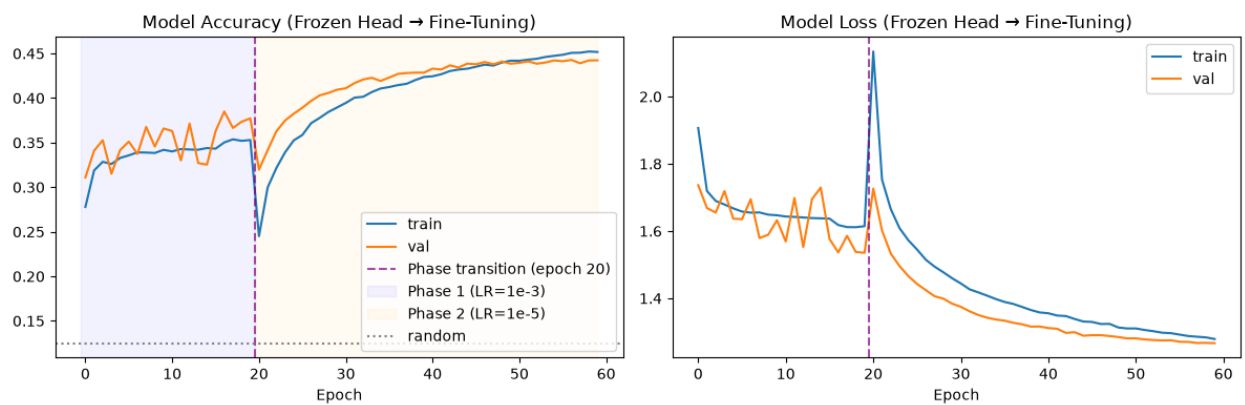


Figure 29: EfficientNetB0 training history

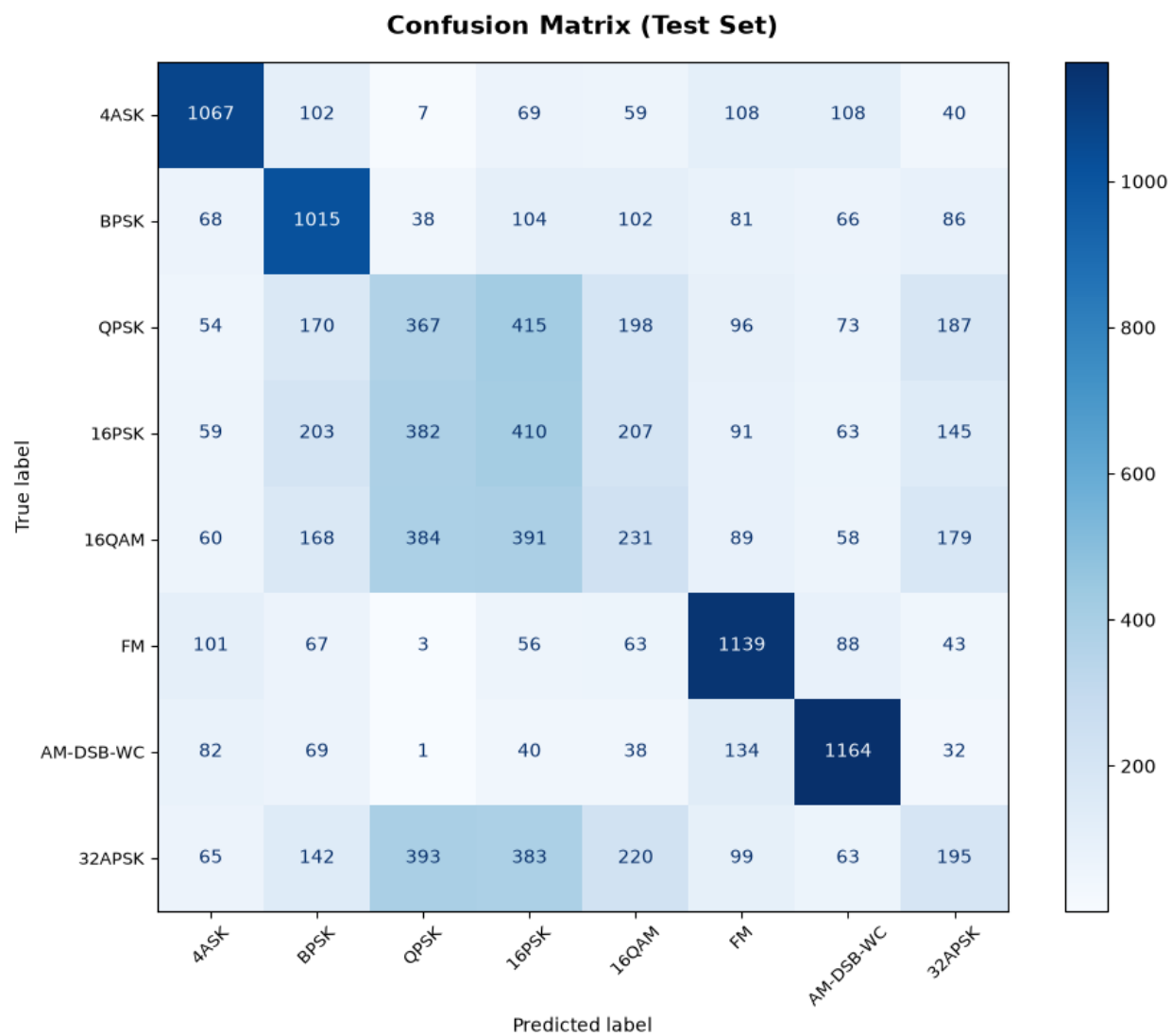


Figure 30: EfficientNetB0 confusion matrix

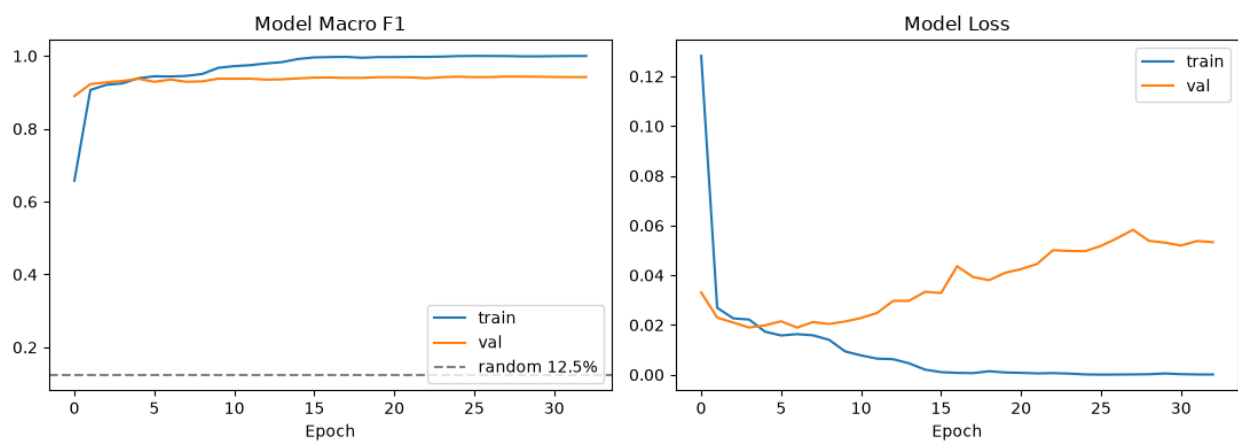


Figure 31: Final model training history

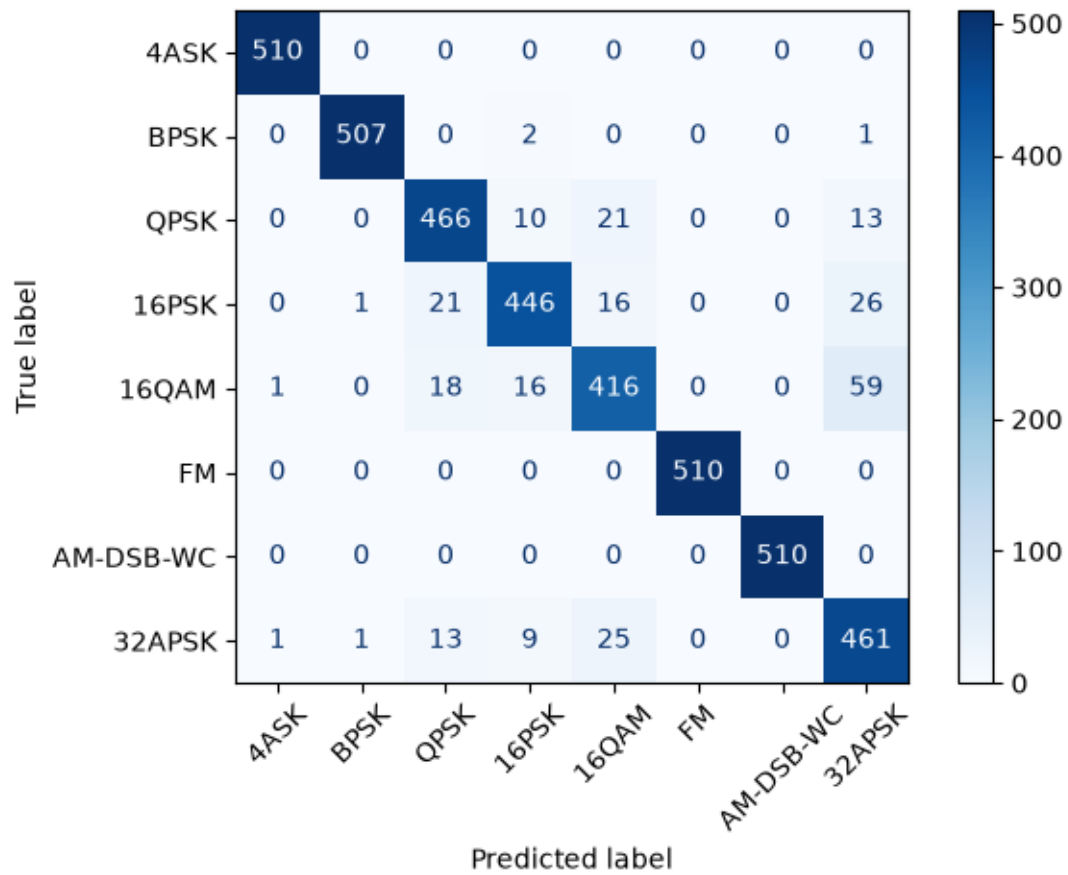


Figure 32: Final model confusion matrix

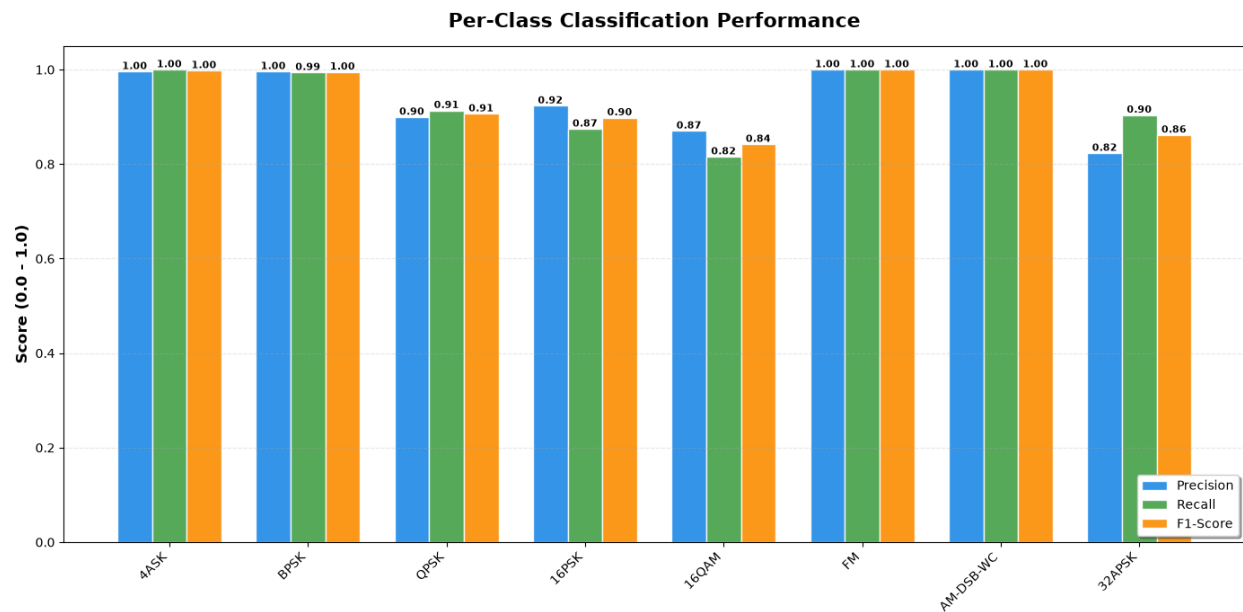


Figure 33: Final model per-class metrics

low-order classes are saturated; the remaining spread is confined to the four PSK, QAM, and APSK classes.

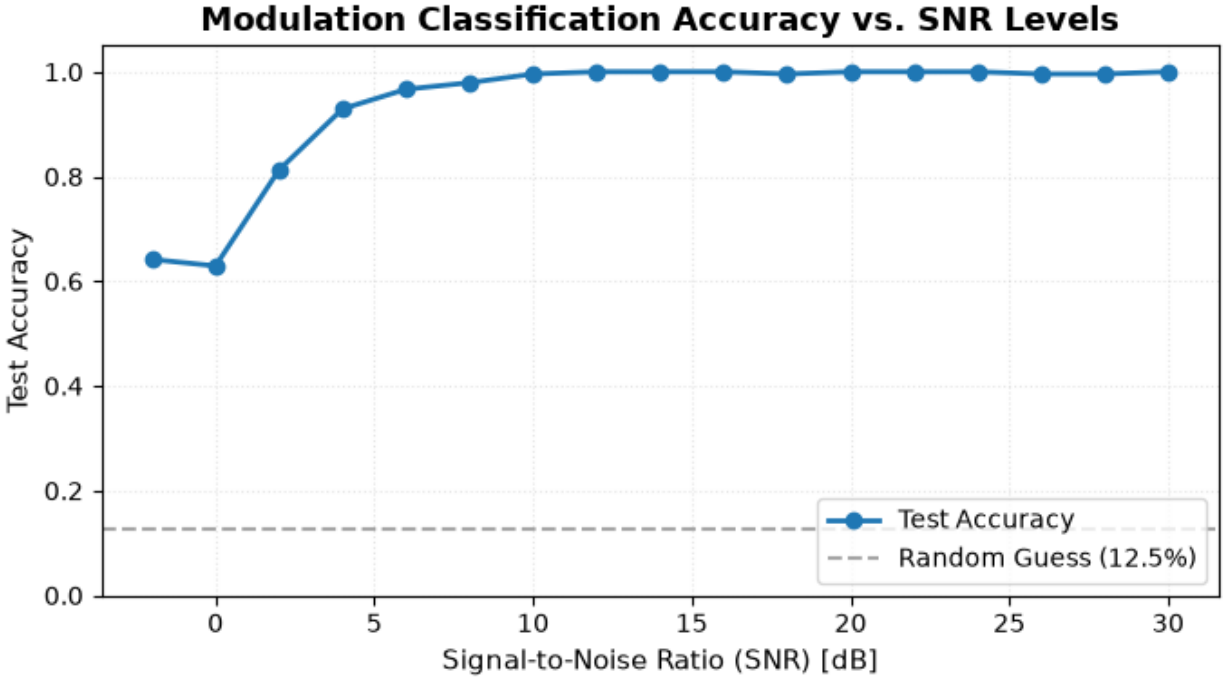


Figure 34: Final model accuracy by SNR

Figure D8. Final five-channel CNN accuracy by SNR. The dashed line is the eight-class random-guess floor; contrast the steep rise with the baseline plateau at 60–67%.

Appendix E - OTA visual evidence

Figures E1–E3 are the three supplied OTA visual outputs, referenced from the Over-the-air experiment subsection of the Evaluation chapter. Each is a five-panel feature-map composite generated by `infer_ota.py` from a different frame of the 94.9 MHz FM capture. They show the exact representation presented to the classifier.

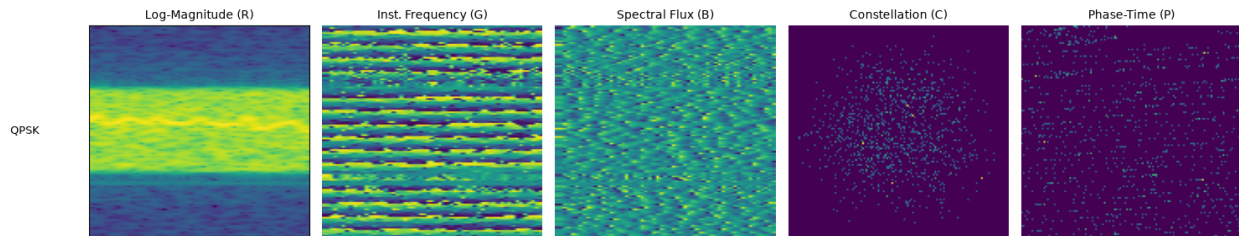


Figure 35: OTA frame 243: five-channel input

Figure E1. OTA frame 243, classified by the model as QPSK.

Figure E2. OTA frame 1783, showing a second sampled input from the same FM capture.

Figure E3. OTA frame 2108, showing a third sampled input used for visual sanity checking.

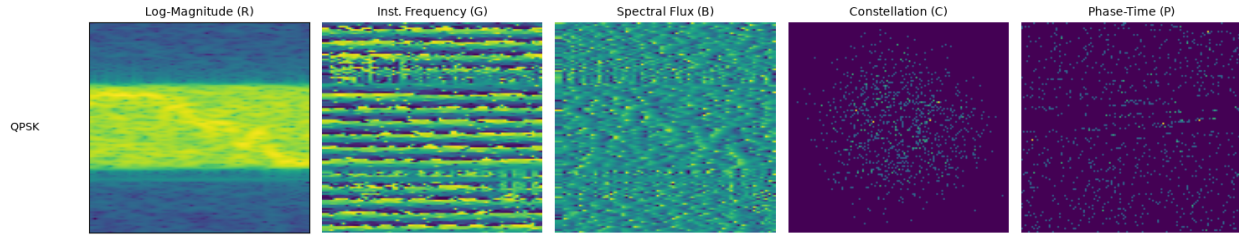


Figure 36: OTA frame 1783: five-channel input

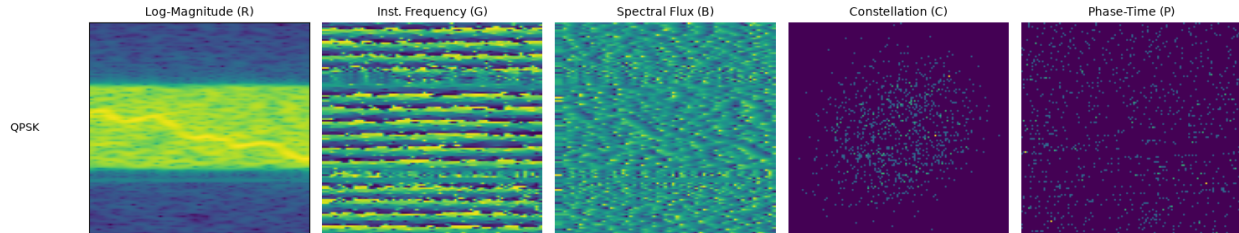


Figure 37: OTA frame 2108: five-channel input

These images are diagnostic evidence, not additional labelled test data. They support the design explanation by making the five input channels inspectable, and they support the evaluation by showing that a confident but incorrect QPSK result came from real OTA inputs rather than from a fabricated example.

Appendix F - Design-evolution figures

These figures document intermediate representation experiments rather than final test results. Figure F1 compares per-class channel patterns at high SNR. Figure F2 shows the three-channel representation used before adding geometry. Figure F3 compares group delay with spectral flux. This experiment motivated retaining spectral flux.

Figure F1. Intermediate per-class channel validation at +30 dB.

Figure F2. Intermediate three-channel representation before the constellation and phase-time channels were added.

Figure F3. Intermediate comparison of group delay and spectral flux. Spectral flux was retained because it produced more structured class evidence.

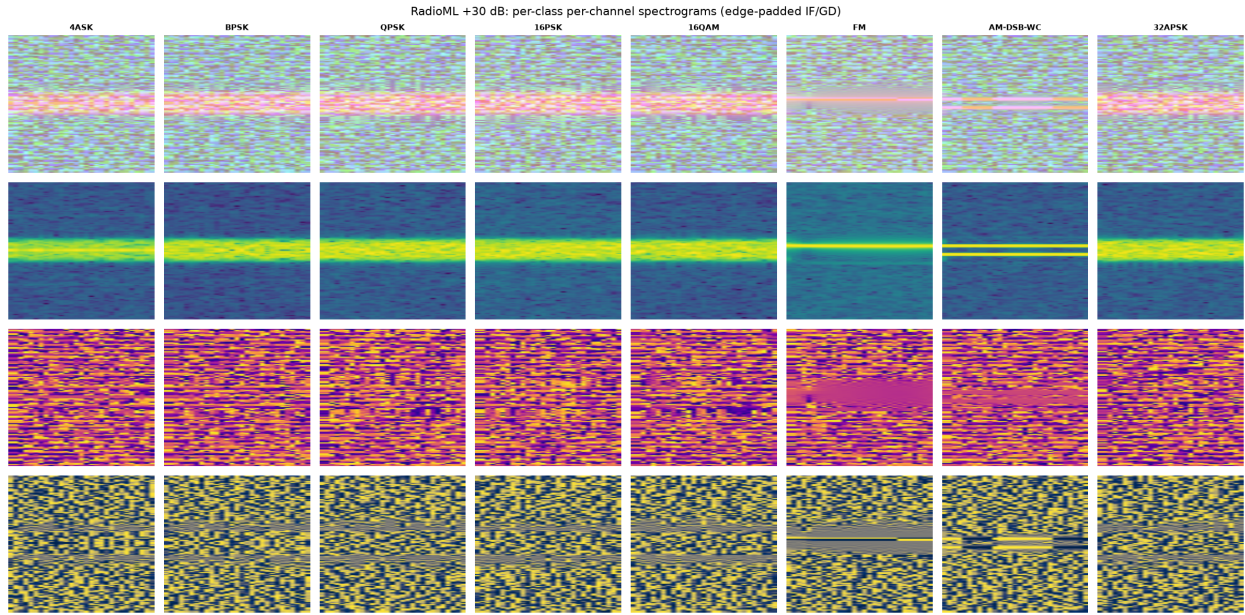


Figure 38: Per-class channel validation

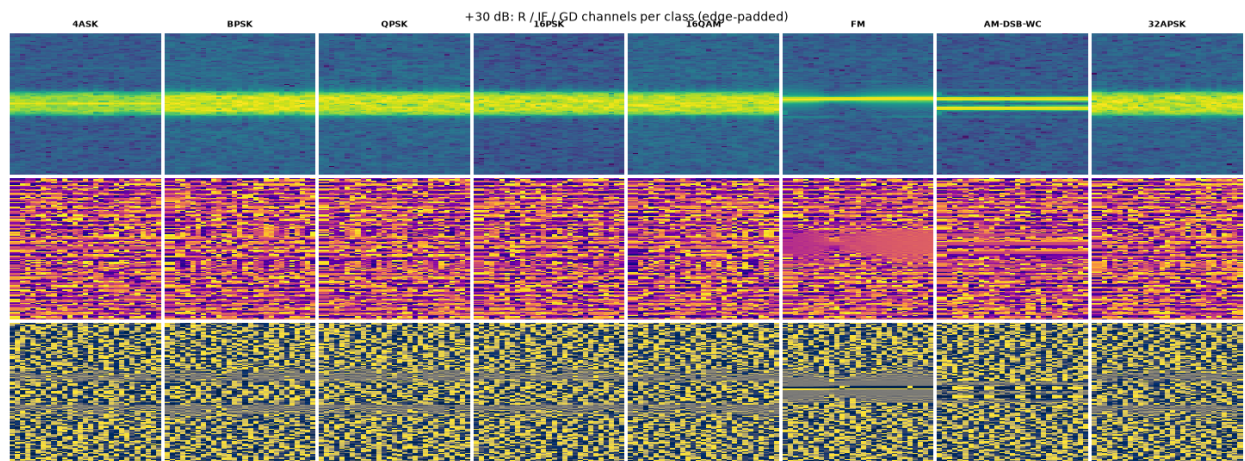


Figure 39: Three-channel representation

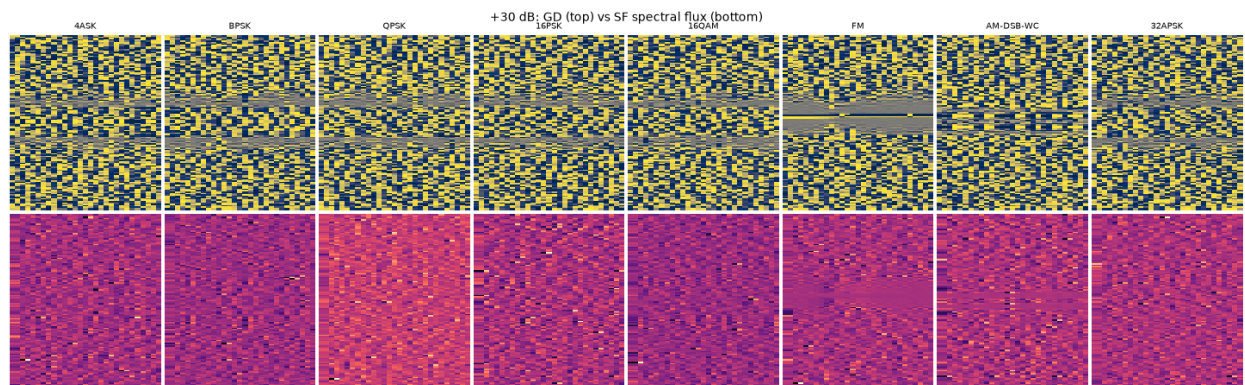


Figure 40: Group delay versus spectral flux